

ADVERSARIAL ATTACKS LEVERAGE INTERFERENCE BETWEEN FEATURES IN SUPERPOSITION

Anonymous authors
Anonymous submission

ABSTRACT

ABSTRACT

Fundamental questions persist about when and why adversarial examples (AExs) arise in neural networks (NNs), and existing accounts typically frame them either as artifacts of irregular decision landscapes or as consequences of sensitivity to non-robust input features. This paper advances a stronger mechanistic account: adversarial vulnerability can arise directly from *efficient* information encoding in NNs. We show that superposition – the representation of more features than available dimensions – creates latent feature arrangements whose interference patterns give adversaries a concrete mechanism to exploit. Across controlled settings, adversarial perturbations reliably leverage interference between superposed features, making attack structure predictable from the learned feature geometry. The framework explains two established phenomena through the same mechanism: transferability of attacks between models trained under similar regimes and class-specific vulnerability patterns. In synthetic settings with precisely controlled superposition, we establish that superposition *suffices* to create adversarial vulnerability. We then show that the same qualitative mechanism persists in a vision transformer (ViT) trained on CIFAR-10. Together, these results identify adversarial vulnerability as a byproduct of representational compression, rather than only a flaw in learning or a property of non-robust inputs. We release our implementation through a software repository.

1 INTRODUCTION

Despite extensive research on AExs (Szegedy et al., 2014; Goodfellow et al., 2014), the field still lacks a mechanistic account that explains their fundamental origin. Minimal perturbations can drastically alter model predictions, yet existing explanations do not fully predict which perturbations succeed, why attacks transfer between models, or how to design defences grounded in the internal computation of the model.

Existing explanations broadly fit into two camps (Nakkiran, 2019). The *bug* perspective attributes vulnerability to learning failures (Schmidt et al., 2018; Nakkiran, 2019) or to geometric properties of high-dimensional decision boundaries (Fawzi et al., 2016). Work in this domain often provides statistical bounds on properties such as capacity (Bubeck & Sellke, 2021) or Lipschitz constraints (Hein & Andriushchenko, 2017), but it does not identify the specific perturbation structure that succeeds. The *feature* perspective argues that AExs exploit predictive but non-robust statistical patterns in data (Ilyas et al., 2019), shifting the debate toward how data semantics relate to human perception. This view treats non-robust features primarily as fixed properties of the data rather than as outcomes of network encoding strategies. Neither perspective directly explains how representational constraints interact with data semantics to determine adversarial behaviour.

We bridge this divide with a mechanistic interpretability account showing that adversarial vulnerability emerges from the interaction between architectural constraints and data semantics (*i.e.* human interpretable properties of the data). We demonstrate that AExs can arise from interference between learned representations in NNs. Adversarial attacks systematically exploit superposed latent structure – a mechanism that normally expands representational capacity – to craft perturbations that manipulate model outputs. Our analysis identifies a concrete pathway: input correlations constrain feature

arrangements, these arrangements determine interference patterns, and those interference patterns dictate attack characteristics and transferability. This framework turns adversarial vulnerability from a largely descriptive phenomenon into one whose perturbation structure and transfer behaviour can be predicted from latent geometry.

Our account of AExs builds on recent theories of how networks encode and process information, namely the linear representation hypothesis (LRH) (Park et al., 2024) and the theory of superposition (Elhage et al., 2022). The LRH posits that *input features*—fundamental abstractions of data—are represented as linear directions in a network’s representation space. Superposition further posits that NNs can represent far more features than they have neurons by efficiently packing multiple features into shared dimensions, gaining capacity at the cost of interference. This interference means that perturbing one feature can predictably affect others through non-orthogonal latent directions. We therefore test whether this interference is not merely an interpretability nuisance, but a mechanism that adversarial attacks actively exploit. Our primary contributions are:

- We introduce a mechanistic framework that reconciles architectural constraints with feature properties in adversarial vulnerability. Using simple models, we demonstrate that data properties induce specific superposition geometries that directly determine realised adversarial perturbations. We show that interference patterns explain attack transferability and class-specific susceptibility.
- We show that these findings persist in a ViT trained on CIFAR-10 with an engineered bottleneck layer that induces superposition.
- We establish that superposition is sufficient, but not necessary, for adversarial vulnerability, and identify algorithmic brittleness as a distinct vulnerability mechanism.
- We demonstrate that mechanistic understanding of learned representations enables informed adversarial attack construction, supporting our case for semantically informed defences.

2 BACKGROUND

We rely on two representational tools that make our mechanism precise: the LRH and superposition. Formally, let $\mathbf{x} \in \mathcal{X}$ denote the input to a NN and $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ the activation vector of the l -th layer.

Linear representation hypothesis (LRH). The LRH states that NNs represent many computational variables, including semantic properties of their inputs, as linear directions in activation space, making those variables usable abstractions for reasoning (Park et al., 2024; Guerner et al., 2023). Linear representations are a powerful default because linear separability lets networks recognise and manipulate features directly, while dot products with subsequent layer weights efficiently process such directional features. A growing body of evidence supports this view (Gurnee et al., 2023; Park et al., 2024). Let $\mathcal{C} = \{c_1, \dots, c_M\}$ denote a set of M semantically meaningful latent features (*e.g.* concepts like “presence of shape,” or “indoor vs. outdoor”). Formally:

Definition 1 (Linear Representation Hypothesis (LRH)) *A neural network layer with activations $\mathbf{h}^{(l)} \in \mathbb{R}^{d_l}$ satisfies the LRH if it represents the latent features $\mathcal{C} = \{c_1, \dots, c_M\}$ as linear directions $\{\mathbf{v}_j\}_{j=1}^M \subset \mathbb{R}^{d_l}$ such that:*

$$\mathbf{h}^{(l)}(\mathbf{x}) \approx \sum_{j=1}^M a_j(\mathbf{x}) \mathbf{v}_j$$

where $a_j(\mathbf{x}) \in \mathbb{R}$ represents the activation magnitude of feature c_j , and $\mathbf{v}_j$ is the corresponding linear direction. The features are **linearly accessible** (Costa et al., 2025): inputs $\mathbf{x}_0, \mathbf{x}_1 \in \mathcal{X}$ that differ mainly in the value of feature c_j while holding other features approximately constant (*i.e.*, $a_j(\mathbf{x}_1) - a_j(\mathbf{x}_0) = k$ and $|a_i(\mathbf{x}_1) - a_i(\mathbf{x}_0)| < \lambda$ for all $i \neq j$ and some small $\lambda > 0$) satisfy:

$$\mathbf{h}^{(l)}(\mathbf{x}_1) - \mathbf{h}^{(l)}(\mathbf{x}_0) \approx k \mathbf{v}_j$$

where k reflects the change in c_j .

Superposition and sparsity. NNs can represent many more latent features than there are available activation dimensions: $M > d_l$. For example, LLMs can reference far more place names than they have residual stream dimensions. Superposition explains this capacity by positing that networks use an overcomplete, non-orthogonal set of feature directions $\{\mathbf{v}_j\}_{j=1}^M$. This strategy leverages the fact that $2^{\Theta(d\epsilon^2)}$ almost orthogonal vectors ($< \epsilon$ cosine similarity) can be represented in d -dimensional

space (Tkocz, 2012), and that sparse vectors can be accurately recovered after projection into lower-dimensional spaces (Elhage et al., 2022; Bereska & Gavves, 2024; Sawmya et al., 2025). This efficient encoding creates two central challenges. First, *polysemanticity* (Scherlis et al., 2022; Lecomte et al., 2023) arises because individual neurons contribute to multiple features, so a neuron’s activation no longer corresponds to a single interpretable concept. Second, non-orthogonal feature directions create *interference* between features: activating one feature partially activates others. Networks can reduce this ambiguity through non-linear operations (e.g. ReLU, softmax) that help disambiguate superposed features (Gurnee et al., 2023).

Definition 2 (Superposition Hypothesis) *A network layer represents features in superposition if:*

1. *The number of latent features exceeds layer dimensionality: $M > d_l$*
2. *There exists non-orthogonal feature directions: $\exists i, j$ with $i \neq j$ such that $\mathbf{v}_i \cdot \mathbf{v}_j \neq 0$*
3. *Latent feature activations are sparse: $\mathbb{E}_{\mathbf{x}}[\|\mathbf{a}(\mathbf{x})\|_0] \ll M$, where $\mathbf{a}(\mathbf{x}) = [a_1(\mathbf{x}), \dots, a_M(\mathbf{x})]^T$*

The network trades representational capacity against feature interference by packing more features than dimensions.

Adversarial attacks. We use the standard definition of adversarial perturbations and attacks following (Szegedy et al., 2014; Costa et al., 2024):

Definition 3 (Adversarial attack) *Let $\mathbb{B}_p(\mathbf{x}, \epsilon)$ denote the ϵ -ball around $\mathbf{x} \in \mathcal{X}$ in the ℓ_p -sense, e.g. $\mathbb{B}_\infty$ is the ℓ_∞ -ball. Any small δ that p -perturbs $\mathbf{x}$ to create an **adversarial input** $\mathbf{x}' \in \mathbb{B}_p(\mathbf{x}, \epsilon)$ such that $f(\mathbf{x}') \neq f(\mathbf{x})$ (untargeted) or $f(\mathbf{x}') = y_{\text{target}}$ (targeted) is known to be an **adversarial attack**. ℓ_∞ - or ℓ_2 -norm attacks are created via the projected gradient descent (PGD) (detailed in Sec. A).*

Definition 4 (Adversarial robustness) *A classifier $g_w(\cdot)$ with parameters w is said to be **robust** at $\mathbf{x}$ with radius $\epsilon > 0$, if $g_w(\mathbf{x}') = g_w(\mathbf{x}) \forall \mathbf{x}' \in \mathbb{B}(\mathbf{x}, \epsilon)$.*

3 SUPERPOSITION GEOMETRY DETERMINES ADVERSARIAL ATTACKS

We first isolate the central mechanism: adversarial attacks exploit interference created by superposed latent features. Our experiments test three concrete questions:

1. *Do adversarial perturbations exploit the interference between superposed features?*
2. *Do correlations in the input shape the geometric arrangement of superposed latent features?*
3. *Can shared latent geometry explain why attacks transfer between independently trained models?*

Experimental setup. To answer these questions decisively, we construct a synthetic task that instantiates established LRH and superposition phenomena (Park et al., 2024; Elhage et al., 2022) in a precisely controlled setting. The task is built to: (1) provide an intuitive classification setting for studying AExs; (2) represent class concepts as linear directions per the LRH; (3) induce controlled superposition between these latent features; and (4) preserve a priori knowledge of how inputs map to superposed features, enabling direct tests of the adversarial mechanism.

We partition input $\mathbf{x} \in \mathbb{R}^d$ into k equally-sized groups $\mathbf{x} = [\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(k)}]$, where $\mathbf{x}^{(j)}$ contains the p input features associated with class j . The label is the group with the largest sum: $y = \operatorname{argmax}_{j \in \{1, \dots, k\}} \sum_{i=1}^p x_i^{(j)}$. Thus each $x_i^{(j)}$ contributes evidence for class j , while the p features capture distinct class attributes. Following Elhage et al. (2022), we sample $x_i^{(j)} \sim \text{Uniform}(0, 1)$ with sparsity S (probability of setting to zero).

A two-layer network compresses the input into m dimensions through a bottleneck encoder, $\mathbf{h} = \text{ReLU}(\mathbf{W}_e \mathbf{x} + \mathbf{b}_e) \in \mathbb{R}^m$ (where $m < k < d$), and reconstructs class logits through a decoder, $\mathbf{z} = \mathbf{W}_d \mathbf{h} + \mathbf{b}_d \in \mathbb{R}^k$. Our primary setup uses cross entropy (CE) loss without ReLUs or biases because this clean configuration makes the geometry explicit, while the softmax limits interference by exponentially suppressing contributions from interfering latent features. To verify that the mechanism is not an artefact of this simplified choice, we replicate the findings in a second setup with mean squared error (MSE) loss, ReLUs, and biases. The network must compress information from $d = k \times p$ dimensions to $m < k$ dimensions and still recover enough information to identify the class with the largest sum.

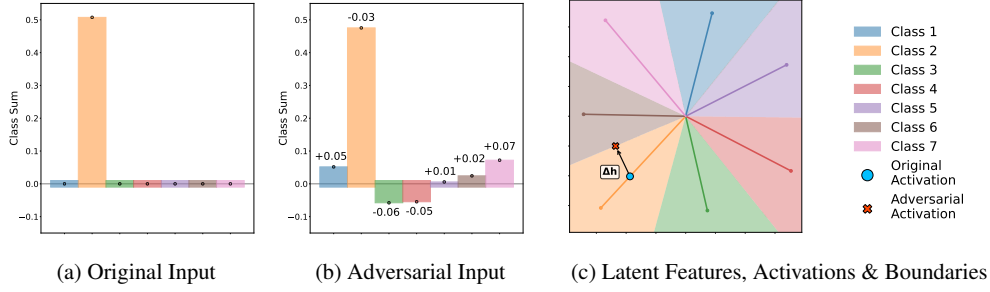


Figure 1: **An adversarial attack that exploits superposition geometry** ($k = 7, m = 2$). (a) The original sample. (b) The adversarially perturbed sample, whose ground truth remains unchanged but is misclassified. The configuration of latent representations determines the sign and magnitude of the input perturbation. (c) The original and adversarial sample in activation space. The arrows are the column vectors of $\mathbf{W}_e$, the latent representations of the input features.

Unlike non-synthetic tasks, where nuanced features must be discovered and interpreted, this setup gives ground-truth class labels for the latent features. We can therefore recompute the true class after any perturbation by recalculating group sums and define an AEx by two explicit conditions:

1. The model’s prediction under perturbation changes (*e.g.* from class A to B)
2. The true class remains unchanged (*e.g.* still class A)

This criterion cleanly separates genuine AExs from perturbations that change the ground truth itself (*i.e.* change which group has the highest sum). We generate attacks using ℓ_∞ - and ℓ_2 -norm PGD.

Evaluation metrics. We treat the columns of the encoder weight matrix $\mathbf{W}_e$ as the latent representations $\mathbf{v}_i$ for each input feature x_i . Because input feature $x_i^{(j)}$ contributes exclusively to class j , these $\mathbf{v}_i$ (for $i \in \text{class } j$) align into class-specific directions. This direct correspondence lets us trace exactly how perturbations to inputs of a specific class, $\mathbf{x}^{(j)}$, move the representation in latent space. For successful PGD-generated AExs $\mathbf{x}_{adv} = \mathbf{x}_{orig} + \delta$, we measure:

1. **Input perturbation profile (IPP):** The sign and magnitude of each δ_i , describing how different input features are perturbed in an attack.
2. **Latent attack alignment:** The similarity $\Delta \mathbf{h} \cdot \mathbf{v}_c$ between the *latent attack vector* $\Delta \mathbf{h} = \mathbf{h}_{adv} - \mathbf{h}_{orig}$ and class c ’s representation.
3. **Attack transferability:** Success rate of attacks generated on one model when applied to another.
4. **Robust accuracy:** The fraction of examples that remain correctly classified after perturbation δ with $\|\delta\|_p \leq \epsilon$ to a sample of inputs.

3.1 RESULTS

Do attacks exploit interference between latent features in superposition? An attack must move a sample across a decision boundary to change its class, and our central question is what determines the required input perturbation. Two standard intuitions are insufficient: the *feature intuition* predicts that attacks simply increase target-class input features, while the *bug intuition* treats adversarial perturbations as unpredictable modifications for which feature representations provide little guidance.

Fig. 1 shows a representative AEx: an input from Class 2 (orange) is perturbed according to its IPP and then misclassified as Class 6 (brown). The perturbation does not simply increase target-class features, ruling out the naive feature intuition. It is also not arbitrary. Instead, δ follows a precise pattern determined by the configuration of latent features. This establishes the relevant intuition for the rest of the paper: *attacks are mediated by latent geometry*.

Quantitatively, the IPP strongly tracks latent attack alignment. The sign and magnitude of perturbations $\delta^{(j)}$ are determined by how each class representation $\mathbf{v}^{(j)}$ aligns with the latent attack vector $\Delta \mathbf{h}$. Positively aligned representations ($\mathbf{v}^{(j)} \cdot \Delta \mathbf{h} > 0$) receive proportional amplification in input space, while negatively aligned

Table 1: PGD-discovered attacks align with theoretically optimal perturbations across configurations. Results show mean $\pm$ std over 1000 attacks per condition. All p -values are below 10^{-10} .

k	m	Cosine Sim. (PGD vs Theory)	Cosine Sim. (Random Baseline)
6	2	0.97 ± 0.02	0.00 ± 0.02
30	10	0.96 ± 0.00	0.00 ± 0.01
90	30	0.92 ± 0.00	0.00 ± 0.00

representations are attenuated. Thus Fig. 1(b) is not a visual coincidence: the input perturbations are the input-space expression of the dot products in Fig. 1(c), chosen to move the sample across the decision boundary.

To test directly whether PGD exploits superposition geometry, we compare PGD-generated attacks with theoretically optimal perturbations that explicitly leverage superposition (derived in Sec. 3.2). For each configuration (k, m) , we train 5 models with different random seeds and generate 1000 PGD attacks per model, then compute the cosine similarity between each successful perturbation and its corresponding optimum. We define a random baseline by generating perturbations with matching ℓ_2 norm and measuring their similarity to the theoretical optimum. Tab. 1 shows near-perfect alignment between PGD and optimal attacks across dimensionalities. We filter out samples with ℓ_2 norm less than ϵ to retain meaningful perturbations. One-sample t-tests against the random baseline yield $p < 10^{-10}$ for all configurations, decisively showing that PGD attacks match theoretical predictions rather than chance directions. Since the optimal attacks exploit interference and PGD aligns with these optima, PGD is exploiting superposition interference.

Finding: Adversarial attacks systematically exploit interference between superposed features. Successful PGD attacks are predictable given the specific superposition geometry, rather than random.

Do input correlations determine latent feature geometry? We next test whether input correlations determine the geometry of latent features. By systematically manipulating correlation structure in the training data, we measure how strongly these correlations constrain the degrees of freedom in learned representations. We use three correlation patterns with increasing constraint strength. Under the *uncorrelated* condition, features are sampled independently as before. For *paired* correlations, each feature in class i is coupled with a feature in class j , so one activates whenever its pair activates. For *global* correlations, each sample has a random phase that determines a cyclic activation pattern: adjacent classes in index have similar activation probabilities and frequently co-activate, while more distant classes rarely co-activate.

Fig. 2 shows that these correlations determine the arrangement of $\mathbf{v}^{(j)}$. We quantify geometric similarity by comparing pairwise cosine similarity matrices between all feature pairs in each model and then measuring the correlation between these matrices across random seeds. Higher correlation indicates more similar geometric arrangements. For each correlation condition, we tested 25 seed pairs per condition. For the $m = 2, k = 7$ setup, geometric similarity was 0.23 ± 0.14 (uncorrelated), 0.47 ± 0.32 (paired), and 0.88 ± 0.06 (global) (mean $\pm$ std). The relationship is monotonic: uncorrelated data permits highly variable geometries across seeds, paired correlations partially constrain the arrangements, and global correlations force near-identical geometries. Pairwise two-sample t-tests with Bonferroni correction confirm significance between all three modes ($p < 10^{-3}$ for all comparisons, $\alpha = 0.05$ corrected for 3 comparisons). Results across various (k, m) configurations are provided in Sec. C.3.

The mechanism is *interference avoidance*: frequently co-activating features are arranged to minimise mutual interference. Stronger correlations therefore impose stronger geometric constraints: uncorrelated data allows any arrangement that separates classes, paired correlations fix relative positions of coupled features, and global correlations determine a unique geometry up to rotation.

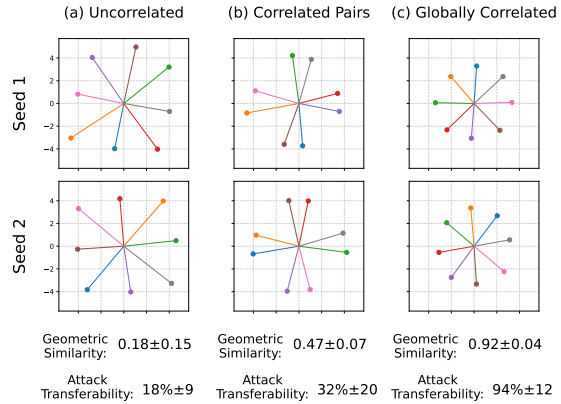


Figure 2: Stronger input correlations impose more consistent geometries across initialisations, increasing attack transferability from 18% (uncorrelated) to 94% (global). Error bars show standard deviations.

Finding: Input correlations constrain feature geometry. Stronger correlations reduce geometric degrees of freedom, forcing different initialisations to converge to similar arrangements.

Does shared geometry explain attack transferability? Having established that input correlations induce consistent latent geometries across model initialisations, we test whether the resulting shared interference patterns explain attack transferability. If attacks are tailored to specific interference patterns, they should transfer successfully precisely when models learn similar geometric arrangements.

Using the same seed pairs as above, we generate adversarial attacks on source models and evaluate their success on target models within each correlation condition (25 transfer measurements per condition). All models achieve $> 95\%$ clean accuracy. Transfer rates track geometric similarity strongly. For the $k = 7, m = 2$ configuration (Fig. 2), globally correlated data yields $94\% \pm 12\%$ attack transfer (mean $\pm$ std), uncorrelated data yields only $18\% \pm 9\%$ transfer, and paired correlations produce the intermediate rate of $32\% \pm 20\%$.

Each perturbation component amplifies or suppresses features, producing constructive interference that pushes representations across decision boundaries. When the same perturbation is applied to a model with different feature arrangements, the features that previously interfered constructively instead cancel or interfere destructively, so the attack fails.

Finding: Attack transferability is governed by shared interference patterns.

Does reducing superposition suppress these attacks?

If superposition creates vulnerability through interference, then reducing superposition should suppress the corresponding attacks. We test this prediction with three interference-limiting experiments. First, we set $m = k$, allowing the network to represent each class direction $\mathbf{v}^{(j)}$ orthogonally. In this configuration, any perturbation that changes the model’s prediction also changes the ground-truth class: moving a sample from class A to class B requires making the class B feature sum exceed the class A sum, genuinely transforming it into a class B sample. We find zero successful adversarial examples across 1000 attempts at every ϵ tested. Second, we isolate superposition’s effect from capacity by fixing m and varying k . Robust accuracy decreases monotonically with superposition pressure k/m (App. Tab. 5), demonstrating that vulnerability scales with interference degree. Third, we test whether attacks concentrate on superposed features by constraining one class’s latent vector $\mathbf{v}^{(\perp)}$ to remain orthogonal to all others during training. When generating attacks between classes that remain in superposition (e.g. from class a to b where $\mathbf{v}_a \not\perp \mathbf{v}_b$), the inputs corresponding to the orthogonal class remain unperturbed (Fig. 3).

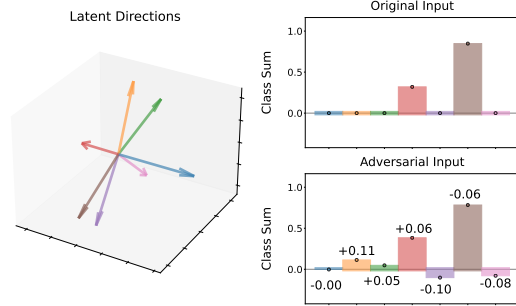


Figure 3: An adversarial attack (from class 5 to class 3) leaves the input features for an orthogonally represented class (class 1) unperturbed.

Finding: Adversarial attacks use their budget to exploit those features in superposition.

3.2 FORMAL ANALYSIS

We now formalise the mechanism behind these empirical results. In our linear setting, we derive exact relationships between superposition interference and adversarial vulnerability, moving the claim beyond observational correlation. We use the notation from our experimental setup: $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ induces superposition with columns $\mathbf{v}_i$, and the decoder corresponds to the encoder transpose (as empirically observed). The logit for class j is $z_j = \mathbf{v}_j^\top \mathbf{h}$.

Proposition 1. *The optimal input perturbations δ that maximise movement from class j to class k under constraint $\|\delta\|_2 = \epsilon$ satisfy $\delta \propto \mathbf{W}_e^\top \mathbf{n}$, where $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ is the normal to the decision boundary between classes.*

Proof Sketch. To move from class j to k , we maximise the logit margin $z_k - z_j = \mathbf{n}^\top (\mathbf{h} + \Delta \mathbf{h})$, where $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ and $\Delta \mathbf{h} = \mathbf{W}_e \delta$. This becomes $\max_{\|\delta\|_2=\epsilon} \delta^\top \mathbf{W}_e^\top \mathbf{n}$. By Cauchy-Schwarz, this is maximised when $\delta \propto \mathbf{W}_e^\top \mathbf{n}$. Full proof in Sec. B. $\square$

Corollary 1 (Interference Drives Vulnerability) *The adversarial perturbation magnitude for feature i is $|\delta_i| \propto |\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)|$, directly proportional to the differential interference between feature i and the class representations.*

This gives the mechanism by which superposition creates adversarial vulnerability. Each input feature i is perturbed in proportion to its interference with the class representations. Under superposition, non-orthogonality makes even semantically unrelated features interact with the class decision, and adversarial perturbations exploit those dependencies to manipulate outputs.

Proposition 2. *Models with feature representations related by orthogonal transformation $\mathbf{Q}$ (where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$) share identical optimal attack directions in input space.*

Proof Sketch. Under transformation $\mathbf{v}'_i = \mathbf{Q} \mathbf{v}_i$, optimal perturbations remain invariant: $\delta'_i \propto (\mathbf{v}'_i)^\top (\mathbf{v}'_k - \mathbf{v}'_j) = \mathbf{v}_i^\top \mathbf{Q}^\top \mathbf{Q} (\mathbf{v}_k - \mathbf{v}_j) = \mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j) = \delta_i$. Full proof in Sec. B. $\square$

Together with our empirical findings, these propositions establish the paper’s core **mechanistic pathway**. Specifically: (i) input correlations constrain feature arrangements in superposition, (ii) these geometric arrangements determine interference patterns, (iii) interference patterns dictate optimal perturbations via $\delta \propto \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$ (Proposition 1), and (iv) shared geometric constraints yield similar interference patterns across models, enabling transferability (Proposition 2):

Correlations $\xrightarrow{\text{constrain}}$ **Feature Geometry** $\xrightarrow{\text{determines}}$ **Interference Patterns** $\xrightarrow{\text{enable}}$ **Transferability**

4 ATTACKS IN REALISTIC MODELS

We next show that the mechanism extends beyond synthetic control to more realistic models: a ViT (Dosovitskiy et al., 2020) trained on CIFAR-10 (Krizhevsky, 2009) (with ResNet-18 (He et al., 2016) results in Sec. D.2.1). By inserting a bottleneck layer directly before the final classification output, we force latent class features into superposition and then test the same predictions as in the toy setting: (i) greater superposition should reduce robustness and increase attack transferability, (ii) consistent feature geometry should emerge across initialisations, and (iii) attacks should exploit feature interference rather than merely the bottleneck’s presence.

Setup. We train a ViT architecture (6 layers, patch size 4, residual stream $d = 512$) on CIFAR-10 across five random seeds. After training, we replace the original classification head with a bottleneck: a linear encoder compresses the pre-classification activations to an m -dimensional space, and a linear decoder expands back to the $k = 10$ CIFAR-10 classes. This bottleneck, trained with CE loss and no ReLU (matching Sec. 3), places the k class representations into m -dimensional superposition. During bottleneck training, the base ViT weights remain frozen. We vary the bottleneck dimension $m \in \{2, 3, 5, 10\}$ to directly control superpositional compression. We generate AEs for each bottlenecked model using both ℓ_∞ - and ℓ_2 -norm PGD. We use a small step size and many iterations, stopping when an attack successfully changes the model’s prediction (full details in Sec. D.1). Robust accuracy is normalised by clean accuracy to account for performance differences across bottleneck capacities. Attack transferability is measured as defined in Sec. 3.

Results. The base models achieve 81% mean clean accuracy. As expected, bottleneck compression reduces clean accuracy to 54%, 67%, 76%, and 81% for $m = \{2, 3, 5, 10\}$.

Three observations reproduce the toy-model mechanism in this more realistic setting. First, Fig. 4 (right) shows that decreasing the bottleneck dimension m (*i.e.* increasing superposition pressure) both lowers normalised robust accuracy and raises attack transferability across model initialisations. The robustness drop is consistent with denser class-representation packing, which makes small input perturbations more likely to induce widespread interference among compressed features. The transferability increase follows the mechanism established in Sec. 3: stronger superposition reduces the degrees of freedom available for feature geometry, leaving fewer viable arrangements for the class representations. Different initialisations therefore converge to more similar geometries and

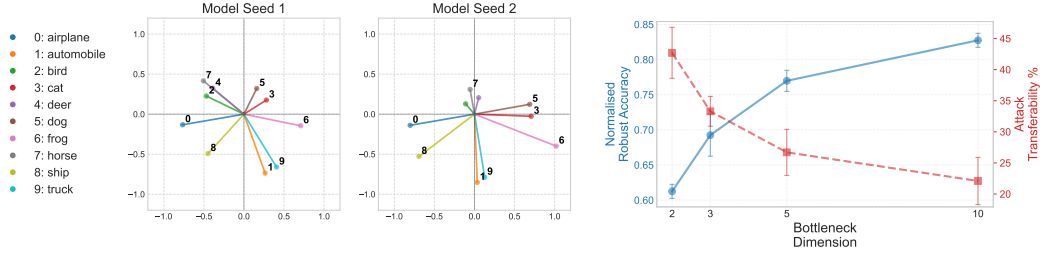


Figure 4: Left: CIFAR-10 class representation structure remains consistent between models across seeds. Right: attack transferability and robust accuracy change systematically as bottleneck dimension increases.

share more interference patterns, which directly increases transferability. See Sec. D.1.1 for results across perturbation magnitudes.

Fig. 4 (left) supports this interpretation by showing that the m -dimensional class representations in the bottleneck exhibit consistent ordering and clustering across models. As in Sec. 3, this consistency indicates that data correlations guide formation of the superposed geometry. The groupings often reflect semantic class similarities (e.g. ‘cat’ and ‘dog’ representations clustering together), strengthening the interpretation that learned feature geometry carries meaningful structure. Finally, AExs generated for a bottlenecked model produce similar relative logit changes when passed through the corresponding base ViT (i.e. $\text{logit}_{\text{base}}(\mathbf{x}_{\text{adv}}) - \text{logit}_{\text{base}}(\mathbf{x}_{\text{orig}})$). The base and bottlenecked models therefore respond coherently to the same perturbations. The bottleneck amplifies vulnerability by increasing compression, but the perturbations are not merely an artefact of the bottleneck itself.

5 ADVERSARIAL VULNERABILITY VIA ALGORITHMIC BRITTLINESS

The previous sections establish that feature superposition can *suffice* for adversarial vulnerability. We now show that it is not *necessary*: even orthogonal feature representations can remain vulnerable when the learned algorithm is brittle. This expands the paper’s central claim from superposition alone to a broader principle: robustness depends on the interaction between representations, architecture, and task semantics. We ask three questions: *how do such attacks work mechanistically, can this understanding be used to construct attacks directly, without gradients, and does naive robustness certification alter the underlying failure mode or merely increase required attack budgets?*

Setup. We train a linear encoder + MLP on one-hot integers (a, b) to predict $(a + b) \bmod P$. As detailed in Sec. E and Nanda et al. (2023), the model discovers a trigonometric algorithm for the task. As part of this learned algorithm¹ the encoder projects inputs onto non-basis-aligned linear representations encoding $\sin(\omega_k a)$ and $\cos(\omega_k a)$ at key frequencies $\omega_k = 2\pi k/P$. The MLP then composes $\cos((a + b)\omega_k)$ via sum-angle identities and reaches 100% accuracy. Crucially, these features are represented *orthogonally* (i.e. not in superposition). We generate AExs by relaxing the one-hot constraint and perturbing only input a using PGD, thereby moving inputs off the training manifold.

Attacks exploit the learned key frequencies. Fourier analysis of successful perturbations δ (App. Fig. 6) shows that their dominant frequencies precisely match the learned algorithm’s key frequencies, ω_k . Attacks transfer 0% of the time to models with different ω_k , e.g. models trained from different seeds. These frequency-aligned perturbations cause misclassification by altering the sinusoidal values read from the encoder features. The perturbation δ transforms one-hot $\mathbf{x}_a$ into a dense vector $\mathbf{x}'_a$. The newly non-zero coordinates contribute additional sinusoidal terms, corrupting the values passed to the MLP and breaking the downstream computation. Because the learned algorithm uses constructive interference of each ω_k at the logits, disrupting a single ω_k is insufficient. Successful attacks must therefore find δ that alters multiple frequency components simultaneously. We provide detailed intuition for this mechanism in Sec. E.1.

¹We refer to the specific computation executed by the network to solve the task as the *model’s learned algorithm*. As this algorithm depends on extracting precise values from learned feature representations (frequencies), it remains *brittle* in the face of small perturbations to the bases. We term this the *algorithmic brittleness*.

Analytic construction of attacks. This mechanism lets us construct gradient-free, *informed* perturbations as composite sinusoidal waves with matching ω_k . On $0 + 0 \bmod P$, these attacks succeed with $\|\delta\|_2=0.22$, close to PGD (0.17, $\approx 1.3\times$ the PGD budget) and far below naive alternatives (single-index 0.97, ℓ_∞ noise 2.04, uniform 6.20). This result confirms that the analysis identifies the operative failure mechanism and shows that representation-aware vulnerability analysis can produce concrete attack recipes, not only post-hoc explanations.

Do defences mitigate this algorithmic vulnerability? We apply certified training against ℓ_∞ perturbations (Müller et al., 2023). For fixed attack budgets, the *robust* accuracy improves from pre-robustified $\rightarrow$ post-robustified as follows: 0% $\rightarrow$ 85% at $\epsilon_\infty = 5e-3$, 0% $\rightarrow$ 96% at $2e-3$, and 76% $\rightarrow$ 100% at $1e-3$. However, under PGD, the robust model still fails through the same frequency-based brittleness, albeit at larger perturbation norms. Certification raises the required budget but does not remove the identified fragility, motivating defences that explicitly target the relevant semantics (here, frequency manipulation) rather than relying only on broad norm bounds.

Relation to the paper’s thesis. This section establishes two points that sharpen the broader narrative. First, superposition is *sufficient but not necessary*: orthogonal representations can still be vulnerable when the learned algorithm is brittle. Second, the architecture–semantics perspective is actionable: it guides both attack construction (via representation-informed perturbations) and defence design.

Although this mechanism is distinct from superposition, it shares a dependence on sparsity assumptions: upstream violations of sparsity (*e.g.* noise or superposed features) can deliver effectively ‘pre-perturbed’ inputs to downstream orthogonal circuits and activate the same frequency-based failure.

6 RELATED WORK

Superposition & latent representations. Superposition research has identified feature interference as a central mechanism in learned representations: Nanda (2023) distinguish *representational* and *computational* superposition, while Gurnee et al. (2023) differentiate *alternating* and *simultaneous* interference. Prior work also shows that data correlations shape representational arrangements: correlated feature pairs become orthogonal (Elhage et al., 2022), correlations drive superposition formation (Chan, 2024), and semantic clustering can emerge from correlated inputs (Prieto et al., 2025a). Beyond superposition, representation geometry is shaped by spectral bias (Rahaman et al., 2019), neural collapse (Kothapalli, 2023), and optimisation objectives (Casper, 2023). Sparse autoencoders (SAEs) have recently made these questions more tractable in vision models by disentangling features in superposition (Lim et al., 2024; Joseph et al., 2025).

Adversarial vulnerability. The adversarial robustness literature is commonly organised around an attack–defence dichotomy. Attacks (Szegedy et al., 2014) use small perturbations to maximise model error, with canonical methods including FGSM (Goodfellow et al., 2014) and PGD (Madry et al., 2018). Robustness work asks models to maintain performance under such attacks. Defences include adversarial training (Madry et al., 2018) and certified training (Wong & Kolter, 2018; Palma et al., 2024), which provide guarantees against perturbations within specified input bounds. Our contribution is complementary: rather than proposing another attack or defence, we identify a representational mechanism that explains why specific attacks succeed and transfer.

The role of input & latent features on adversarial robustness. Several lines of work attribute vulnerability to predictive but brittle ‘non-robust’ input features (Ilyas et al., 2019; Goh, 2019), or to learning shortcuts (Li et al., 2023). Perturbation characteristics also vary across datasets (Maiya et al., 2021). Elhage et al. (2022) suggested a possible link between superposition and AExs, while Casper (2023) debated the connection. A tension between representational efficiency and adversarial robustness has also been observed (Barsbey et al., 2025). Zhang et al. (2021) show that adversarial perturbations push latent representations across decision boundaries, and Ganeshan et al. (2019) find that PGD targets final layers and propose feature-based attacks. Transferability is limited by representation discrepancies between models (Li et al., 2023; Wang et al., 2024); decorrelating features reduces transfer (Wiedeman & Wang, 2022), matching our conclusion that shared interference geometry governs transfer.

7 DISCUSSION & CONCLUDING REMARKS

We have shown that adversarial attacks exploit interference patterns arising from the geometry of superposed features in NNs. Across toy models and ViTs, the evidence supports a coherent mechanism: data properties, especially correlations and sparsity, induce specific superposition geometries; those geometries create interference; and the resulting interference determines predictable adversarial vulnerabilities. This mechanism explains attack transferability and class-specific susceptibility through the same representational structure. The paper therefore reframes adversarial vulnerability as a direct consequence of efficient information encoding via superposition, rather than only a defect of optimisation or an externally fixed property of non-robust inputs.

Limitations & future work. Our strongest causal evidence comes from simplified models, and this scope is deliberate: controlled settings allow us to isolate the mechanism and test its predictions directly. The results offer a clear mechanistic lens for larger models, but the broader adversarial robustness literature indicates that exact vulnerability mechanisms will vary by attack family and model architecture. For example, constrained attack types (*e.g.* universal perturbations, jailbreaks, patch attacks) may not be able to activate the feature combinations required to exploit superposition interference.

Future work should quantify the ‘adversarial cost’ of superposition, connect it to accuracy–robustness trade-offs, and determine how robust training reshapes superposition geometry. Real networks introduce additional complexity: interference occurs between unknown, unlabelled features across multiple layers rather than between engineered class features in a single bottleneck. SAEs (Bricken et al., 2023) provide a concrete route for addressing this challenge in large-scale networks, and the preliminary results in Sec. F indicate that this direction is especially promising.

ETHICS STATEMENT

This work provides a mechanistic analysis of existing adversarial phenomena and is not expected to create specific negative impacts or immediate societal consequences. Our experiments use only publicly available datasets (CIFAR-10) and synthetic toy models, with no human subjects or personally identifiable information. The gradient-based attack methods used in the paper are already well established in the literature. We do not introduce a new attack capability; instead, we clarify why existing attacks can succeed by exposing the relationship between superposition and adversarial vulnerability. This understanding can inform future research that balances capability with robustness and supports the development of more trustworthy AI systems for safety-critical deployment.

REPRODUCIBILITY STATEMENT

We provide source code for all experiments in the supplementary materials. The toy model experiments (Sec. 3) specify the architectures, training procedures, evaluation metrics, and data generation processes. For ViT experiments (Sec. 4), we provide architecture and training details, with additional information in Sec. D.1. The modular arithmetic experiments (Sec. 5) specify the network architecture, data encoding, and learned trigonometric algorithm, with further details in Sec. E.2. Experiments were conducted on an AMD Ryzen Threadripper 3970X 32-Core with 256GB RAM and RTX 3090. Additional ResNet-18 and SAE experiments are documented in Appendices Sec. D.2.1 and Sec. F, respectively.

LLM USAGE STATEMENT

This work used LLMs to assist with prior literature search, automate LaTeX table generation from CSV files (all numerical values were verified), and suggest alternative phrasings that improved manuscript clarity. All substantive research contributions and conclusions remain the original work of the authors.

REFERENCES

- Melih Barsbey, Antônio H Ribeiro, Umut Şimşekli, and Tolga Birdal. On the interaction of compressibility and adversarial robustness. *arXiv preprint arXiv:2507.17725*, 2025.
- Leonard Bereska and Stratis Gavves. Mechanistic interpretability for AI safety - a review. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856.
- Trenton Bricken, Adly Templeton, Joshua Batson, Brian Chen, Adam Jermyn, Tom Conerly, Nick Turner, Cem Anil, Carson Denison, Amanda Askell, Robert Lasenby, Yifan Wu, Shauna Kravec, Nicholas Schiefer, Tim Maxwell, Nicholas Joseph, Zac Hatfield-Dodds, Alex Tamkin, Karina Nguyen, Brayden McLean, Josiah E Burke, Tristan Hume, Shan Carter, Tom Henighan, and Christopher Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.
- Sébastien Bubeck and Mark Sellke. A universal law of robustness via isoperimetry. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan (eds.), *Advances in Neural Information Processing Systems*, pp. 28811–28822, 2021.
- Stephen Casper. EIS IX: Interpretability and adversaries. *AI Alignment Forum*, February 2023.
- Lawrence Chan. Superposition is not “just” neuron polysemanticity. *AI Alignment Forum*, April 2024.
- Joana C Costa, Tiago Roxo, Hugo Proença, and Pedro Ricardo Morais Inacio. How deep learning sees the world: A survey on adversarial attacks & defenses. *IEEE Access*, 12:61113–61136, 2024.
- Valérie Costa, Thomas Fel, Ekdeep Singh Lubana, Bahareh Tolooshams, and Demba Ba. From flat to hierarchical: Extracting sparse representations with matching pursuit. *arXiv preprint arXiv:2506.03093*, 2025.

594 Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hier-
595 archical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*,
596 pp. 248–255, 2009.

597 Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas
598 Unterthiner, Mostafa Dehghani, Matthias Minderer, G Heigold, S Gelly, et al. An image is
599 worth 16x16 words: Transformers for image recognition at scale. In *International Conference on*
600 *Learning Representations*, 2020.

601 Rüdiger Ehlers. Formal verification of piece-wise linear feed-forward neural networks. In Deepak
602 D’Souza and K. Narayan Kumar (eds.), *Automated Technology for Verification and Analysis -*
603 *15th International Symposium, ATVA 2017, Pune, India, October 3-6, 2017, Proceedings*, volume
604 10482 of *Lecture Notes in Computer Science*, pp. 269–286. Springer, 2017.

605 Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec,
606 Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition.
607 *arXiv preprint arXiv:2209.10652*, 2022.

608 Alhussein Fawzi, Seyed-Mohsen Moosavi-Dezfooli, and Pascal Frossard. Robustness of classifiers:
609 from adversarial to random noise. *Advances in neural information processing systems*, 29, 2016.

610 Aditya Ganeshan, Vivek BS, and R Venkatesh Babu. FDA: Feature disruptive attack. In *Proceedings*
611 *of the IEEE/CVF International Conference on Computer Vision*, pp. 8069–8079, 2019.

612 Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever,
613 Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. In *The Thirteenth*
614 *International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*,
615 2025.

616 Gabriel Goh. A discussion of ‘adversarial examples are not bugs, they are features’: Two examples
617 of useful, non-robust features. *Distill*, 2019.

618 Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial
619 examples. *arXiv preprint arXiv:1412.6572*, 2014.

620 Clément Guerner, Anej Svete, Tianyu Liu, Alexander Warstadt, and Ryan Cotterell. A geometric
621 notion of causal probing. *arXiv preprint arXiv:2307.15054*, 2023.

622 Wes Gurnee, Neel Nanda, Matthew Pauly, Katherine Harvey, Dmitrii Troitskii, and Dimitris Bertsimas.
623 Finding neurons in a haystack: Case studies with sparse probing. *Trans. Mach. Learn. Res.*, 2023.

624 Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image
625 recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*,
626 pp. 770–778, 2016.

627 Matthias Hein and Maksym Andriushchenko. Formal guarantees on the robustness of a classi-
628 fier against adversarial manipulation. In *Advances in Neural Information Processing Systems*,
629 volume 30, 2017.

630 Patrick Henriksen and Alessio Lomuscio. Robust training of neural networks against bias field
631 perturbations. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(12):14865–14873,
632 Jun. 2023.

633 Nikolaus Howe, Ian McKenzie, Oskar Hollinsworth, Michał Zajac, Tom Tseng, Aaron Tucker,
634 Pierre-Luc Bacon, and Adam Gleave. Scaling trends in language model robustness, 2025.

635 A. Ilyas, S. Santurkar, D. Tsipras, L. Engstrom, B. Tran, and A. Madry. Adversarial examples are not
636 bugs, they are features. In *Advances in Neural Information Processing Systems 32*, pp. 125–136,
637 2019.

638 Sonia Joseph, Praneet Suresh, Lorenz Hufe, Edward Stevinson, Robert Graham, Yash Vadi, Danilo
639 Bzdok, Sebastian Lapuschkin, Lee Sharkey, and Blake Aaron Richards. Prisma: An open source
640 toolkit for mechanistic interpretability in vision and video, 2025.

- Guy Katz, Clark W. Barrett, David L. Dill, Kyle Julian, and Mykel J. Kochenderfer. Reluplex: An efficient SMT solver for verifying deep neural networks. In Rupak Majumdar and Viktor Kuncak (eds.), *Computer Aided Verification - 29th International Conference*, volume 10426, pp. 97–117. Springer, 2017.
- Vignesh Kothapalli. Neural collapse: A review on modelling principles and generalization. *Trans. Mach. Learn. Res.*, 2023.
- Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. CIFAR-10 dataset.
- Victor Lecomte, Kushal Thaman, Rylan Schaeffer, Naomi Bashkansky, Trevor Chow, and Sanmi Koyejo. What causes polysemanticity? an alternative origin story of mixed selectivity from incidental causes. *arXiv preprint arXiv:2312.03096*, 2023.
- Ang Li, Yifei Wang, Yiwen Guo, and Yisen Wang. Adversarial examples are not real features. *Advances in Neural Information Processing Systems*, 2023.
- Hyesu Lim, Jinho Choi, Jaegul Choo, and Steffen Schneider. Sparse autoencoders reveal selective remapping of visual concepts during adaptation. *arXiv preprint arXiv:2412.05276*, 2024.
- Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Hoagy Cunningham, Thomas Henighan, Adam Jermyn, Andy Jones, Andrew Persic, Zhenyi Qi, T. Ben Thompson, Sam Zimmerman, Kelley Rivoire, Thomas Conerly, Chris Olah, and Joshua Batson. On the biology of a large language model. *Transformer Circuits Thread*, 2025.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards Deep Learning Models Resistant to Adversarial Attacks. In *International Conference on Learning Representations*, February 2018.
- Shishira R. Maiya, Max Ehrlich, Vatsal Agarwal, Ser-Nam Lim, Tom Goldstein, and Abhinav Shrivastava. A Frequency Perspective of Adversarial Robustness. *arXiv*, 2021.
- Mark Niklas Müller, Franziska Eckert, Marc Fischer, and Martin T. Vechev. Certified training: Small boxes are all you need. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- Preetum Nakkiran. A discussion of ‘adversarial examples are not bugs, they are features’: Adversarial examples are just bugs, too. *Distill*, 2019.
- Neel Nanda. A comprehensive mechanistic interpretability explainer and glossary, 2023.
- Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023.
- Alessandro De Palma, Rudy Bunel, Krishnamurthy Dvijotham, M. Pawan Kumar, and Robert Stanforth. IBP regularization for verified adversarial robustness via branch-and-bound. *CoRR*, abs/2206.14772, 2022.
- Alessandro De Palma, Rudy Bunel, Krishnamurthy (Dj) Dvijotham, M. Pawan Kumar, Robert Stanforth, and Alessio Lomuscio. Expressive losses for verified robustness via convex combinations. In *The Twelfth International Conference on Learning Representations, ICLR, 2024*.
- Kiho Park, Yo Joong Choe, and Victor Veitch. The linear representation hypothesis and the geometry of large language models. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024.
- L Prieto, E Stevinson, M Barsbey, T Birdal, and P Mediano. Correlations in the data lead to semantically rich feature geometry under superposition. *Mechanistic Interpretability Workshop at NeurIPS 2025*, 2025a.

-
- Lucas Prieto, Melih Barsbey, Pedro A. M. Mediano, and Tolga Birdal. Grokking at the edge of numerical stability. In *The Thirteenth International Conference on Learning Representations, ICLR, 2025b*.
- Aditi Raghunathan, Jacob Steinhardt, and Percy Liang. Certified defenses against adversarial examples. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. OpenReview.net, 2018.
- Nasim Rahaman, Aristide Baratin, Devansh Arpit, Felix Draxler, Min Lin, Fred A. Hamprecht, Yoshua Bengio, and Aaron C. Courville. On the spectral bias of neural networks. In *Proceedings of the 36th International Conference on Machine Learning, ICML 2019, 9-15 June 2019, Long Beach, California, USA*, volume 97 of *Proceedings of Machine Learning Research*, pp. 5301–5310. PMLR, 2019.
- Shashata Sawmya, Linghao Kong, Ilia Markov, Dan Alistarh, and Nir N Shavit. Wasserstein distances, neuronal entanglement, and sparsity. In *The Thirteenth International Conference on Learning Representations, 2025*.
- Adam Scherlis, Kshitij Sachan, Adam S Jermyn, Joe Benton, and Buck Shlegeris. Polysemanticity and capacity in neural networks. *arXiv preprint arXiv:2210.01892*, 2022.
- Ludwig Schmidt, Shibani Santurkar, Dimitris Tsipras, Kunal Talwar, and Aleksander Madry. Adversarially robust generalization requires more data. In *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 5019–5031, 2018.
- C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus. Intriguing properties of neural networks. In *2nd International Conference on Learning Representations, ICLR 2014, 2014*.
- Tomasz Tkocz. An upper bound for spherical caps. *The American Mathematical Monthly*, 119(7): 606–607, 2012.
- Donghua Wang, Wen Yao, Tingsong Jiang, Xiaohu Zheng, Junqi Wu, and Xiaoqian Chen. Improving the Transferability of Adversarial Examples by Feature Augmentation. *arXiv*, 2024.
- Christopher Wiedeman and Ge Wang. Disrupting adversarial transferability in deep neural networks. *Patterns*, 3(5), 2022.
- Eric Wong and Zico Kolter. Provable defenses against adversarial examples via the convex outer adversarial polytope. In *International conference on machine learning*. PMLR, 2018.
- Huan Zhang, Hongge Chen, Chaowei Xiao, Sven Gowal, Robert Stanforth, Bo Li, Duane S. Boning, and Cho-Jui Hsieh. Towards stable and efficient training of verifiably robust neural networks. In *8th International Conference on Learning Representations, ICLR, 2020*.
- Shufei Zhang, Zhuang Qian, Kaizhu Huang, Qiufeng Wang, Rui Zhang, and Xinping Yi. Towards better robust generalization with shift consistency regularization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pp. 12524–12534. PMLR, 2021.

APPENDIX

A GENERATION OF ADVERSARIAL EXAMPLES

We generate AExs with PGD, the standard iterative attack method (Madry et al., 2018). For untargeted attacks, PGD maximises the classifier’s loss $\mathcal{L}(f(\mathbf{x}'), y_{\text{true}})$; for targeted attacks, it minimises $\mathcal{L}(f(\mathbf{x}'), y_{\text{target}})$ using the update rule:

$$\mathbf{x}'^{(k+1)} = \Pi_{\mathcal{S}} \left(\mathbf{x}'^{(k)} \pm \alpha \mathbf{g}_k \right) \quad (1)$$

(+ for maximisation, − for minimisation). Here $\mathbf{x}'^{(0)}$ is the initial perturbed input, α is the step size, $\mathbf{g}_k$ is the (normalised) gradient $\nabla_{\mathbf{x}} \mathcal{L}(f(\mathbf{x}'^{(k)}), y_{\text{class}})$ (y_{class} being y_{true} or y_{target}), and $\Pi_{\mathcal{S}}$ projects onto the ϵ -ball $\mathcal{S} = \{\mathbf{x}' \mid \|\mathbf{x}' - \mathbf{x}\|_p \leq \epsilon\}$ with input-domain clipping. We use the following ℓ_p constraints:

1. ℓ_{∞} : Constraint $\|\delta\|_{\ell_{\infty}} = \max_i |\delta_i| \leq \epsilon$. $\mathbf{g}_k$ is $\text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(\cdot))$. $\Pi_{\mathcal{S}}$ clips each $\mathbf{x}'_i$ to $[\mathbf{x}_i - \epsilon, \mathbf{x}_i + \epsilon]$.
2. ℓ_2 : Constraint $\|\delta\|_{\ell_2} = \sqrt{\sum_i \delta_i^2} \leq \epsilon$. $\mathbf{g}_k$ is $\nabla_{\mathbf{x}} \mathcal{L}(\cdot) / \|\nabla_{\mathbf{x}} \mathcal{L}(\cdot)\|_{\ell_2}$. $\Pi_{\mathcal{S}}$ rescales $\delta = \mathbf{x}' - \mathbf{x}$ if $\|\delta\|_{\ell_2} > \epsilon$ via $\delta \leftarrow \epsilon \cdot \delta / \|\delta\|_{\ell_2}$.

B OMITTED PROOFS & THEORETICAL RESULTS

We make the representational mechanism explicit by proving that optimal perturbations weight each input dimension according to how strongly its corresponding feature aligns with the path to the decision boundary. The analysis focuses on our linear models without ReLU activation functions.

Consider input $\mathbf{x} \in \mathbb{R}^d$ encoded by $\phi(\mathbf{x}) = \mathbf{W}_e \mathbf{x}$ into latent representation $\mathbf{h} \in \mathbb{R}^m$, where the columns of $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ form an overcomplete dictionary $\{\mathbf{v}_i\}_{i=1}^d$ ($m < d$). In our argmax task, the trained encoder-decoder pair empirically uses the encoder transpose as decoder: $\mathbf{W}_d = \mathbf{W}_e^{\top}$. Consequently, the logit for class j is $z_j = \mathbf{v}_j^{\top} \mathbf{h}$, where $\mathbf{v}_j$ is the j -th column of $\mathbf{W}_e$, and the predicted class is $\hat{y} = \arg \max_j z_j$. Input perturbations map to latent perturbations as $\Delta \mathbf{h} = \mathbf{W}_e \delta$, where $\delta = (\delta_1, \dots, \delta_d)^{\top}$ are the perturbation coefficients.

The decision boundary between classes j and k is the set of latent points where their logits are equal: $\mathcal{B}_{jk} = \{\mathbf{h} : z_j = z_k\}$, equivalently $\{\mathbf{h} : (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{h} = 0\}$. The vector $\mathbf{n} = (\mathbf{v}_k - \mathbf{v}_j)$ is the normal to this pairwise boundary and points in the direction that maximally increases the margin $z_k - z_j$. We recall the propositions before giving the proofs.

Proposition 1 (Optimal targeted attack). The optimal input perturbations δ that maximise movement from class j toward class k under constraint $\|\delta\|_2 = \epsilon$ satisfy:

$$\delta \propto \mathbf{W}_e^{\top} \mathbf{n}$$

Proof. To move a sample from class j toward class k , the perturbation must maximise the logit margin $z_k - z_j$. Under perturbation δ , the new margin is:

$$z'_k - z'_j = \mathbf{v}_k^{\top} (\mathbf{h} + \Delta \mathbf{h}) - \mathbf{v}_j^{\top} (\mathbf{h} + \Delta \mathbf{h}) \quad (2)$$

$$= (\mathbf{v}_k - \mathbf{v}_j)^{\top} (\mathbf{h} + \mathbf{W}_e \delta) \quad (3)$$

$$= (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{h} + (\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{W}_e \delta \quad (4)$$

The change in margin is $(\mathbf{v}_k - \mathbf{v}_j)^{\top} \mathbf{W}_e \delta = \delta^{\top} \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j)$. We seek:

$$\max_{\delta} \delta^{\top} \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j) \quad \text{subject to} \quad \|\delta\|_2 = \epsilon$$

Let $\mathbf{g} = \mathbf{W}_e^{\top} (\mathbf{v}_k - \mathbf{v}_j)$. By the Cauchy-Schwarz inequality:

$$|\delta^{\top} \mathbf{g}| \leq \|\delta\|_2 \|\mathbf{g}\|_2 = \epsilon \|\mathbf{g}\|_2$$

Equality is achieved when δ and $\mathbf{g}$ are parallel. Given the constraint $\|\delta\|_2 = \epsilon$:

$$\delta = \frac{\epsilon}{\|\mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)\|_2} \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$$

□

Corollary (Interference drives vulnerability). For a targeted attack from class j to class k , the adversarial perturbation magnitude for input feature i is:

$$|\delta_i| \propto |\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)|$$

where $\mathbf{v}_i$ is the i -th column of $\mathbf{W}_e$. This quantity represents the differential interference between feature i and the class representations.

This identifies the mechanism by which superposition creates adversarial vulnerability. Each input feature i is perturbed in proportion to $\mathbf{v}_i^\top (\mathbf{v}_k - \mathbf{v}_j)$ —the differential interference between feature i and the class representations. Under superposition, non-orthogonality gives even semantically unrelated features non-zero inner products with $(\mathbf{v}_k - \mathbf{v}_j)$, creating exploitable interference patterns. The multi-class setting strengthens this effect because k classes create many pairwise boundaries, each with its own interference pattern. Adversarial perturbations leverage these cross-feature dependencies by manipulating features through their interference with the class representations. This explains the empirical vulnerability: attacks succeed not by directly increasing target-class features, but by exploiting the interference web created by superposition.

Prop. 1 characterises the optimal perturbation direction for moving from class j to k , providing the gradient for maximising the margin $z_k - z_j$. When intervening classes exist (where $z_j < z_i < z_k$ for some class i), following this gradient might cause the model to predict i before reaching target k . Iterative methods like PGD handle this by recomputing gradients at each step—the global attack path emerges from repeated local decisions rather than a single optimisation. Furthermore, while our analysis assumes $\mathbf{W}_d = \mathbf{W}_e^\top$ based on our empirical observations, the framework extends to arbitrary decoders. In the general case, with decoder $\mathbf{W}_d \in \mathbb{R}^{k \times m}$ having rows $\mathbf{w}_j^\top$, the optimal perturbation becomes $\delta \propto \mathbf{W}_e^\top (\mathbf{w}_k - \mathbf{w}_j)$, capturing interference between encoder directions and decoder weights.

Proposition 2 (Attack transferability). Consider encoders ϕ and ψ with basis matrices $\mathbf{W}_e \in \mathbb{R}^{m \times d}$ and $\mathbf{W}'_e \in \mathbb{R}^{m \times d}$ whose columns are related by orthogonal transformation $\mathbf{v}'_i = \mathbf{Q}\mathbf{v}_i$ (where $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$). If both models use their encoder transpose as decoder (i.e., $\mathbf{W}_d = \mathbf{W}_e^\top$ and $\mathbf{W}'_d = (\mathbf{W}'_e)^\top$), then both models have identical optimal input perturbation vectors for any targeted attack from class j to class k .

Proof. For model ϕ , the optimal perturbation from Prop. 1 is:

$$\delta^\phi \propto \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j)$$

For model ψ with transformed columns $\mathbf{v}'_i = \mathbf{Q}\mathbf{v}_i$:

$$\delta^\psi \propto (\mathbf{W}'_e)^\top (\mathbf{v}'_k - \mathbf{v}'_j) = (\mathbf{W}'_e)^\top (\mathbf{Q}\mathbf{v}_k - \mathbf{Q}\mathbf{v}_j)$$

Since $\mathbf{W}'_e = \mathbf{Q}\mathbf{W}_e$ (all columns are transformed), we have:

$$\delta^\psi \propto (\mathbf{Q}\mathbf{W}_e)^\top \mathbf{Q}(\mathbf{v}_k - \mathbf{v}_j) \tag{5}$$

$$= \mathbf{W}_e^\top \mathbf{Q}^\top \mathbf{Q}(\mathbf{v}_k - \mathbf{v}_j) \tag{6}$$

$$= \mathbf{W}_e^\top (\mathbf{v}_k - \mathbf{v}_j) \tag{7}$$

where the last equality uses $\mathbf{Q}^\top \mathbf{Q} = \mathbf{I}$. Thus δ^ϕ and δ^ψ have identical proportionality. Under the same norm constraint $\|\delta\|_2 = \epsilon$, we have $\delta^\phi = \delta^\psi$. □

Prop. 2 explains why attacks transfer between models trained under similar regimes. When models learn feature representations that differ only by orthogonal transformation—equivalently, the same geometry in different orientations—they share the same vulnerability pattern in input space. The orthogonal transformation preserves all feature inner products and therefore preserves the interference patterns that attacks exploit. This accounts for our empirical finding that models trained on data with the same correlations exhibit high attack transferability: they discover similar feature geometries up to rotation and consequently share adversarial vulnerabilities.

C TOY MODEL EXPERIMENTS

This section provides supplementary details, extended results, and additional discussion for the toy model experiments in the main paper. We report model accuracies across a wider parameter range than the main text, clarifying how capacity and data characteristics such as sparsity shape learning and determine when feature superposition appears. We then provide additional visual examples corresponding to Fig. 1, showing that the attack mechanism remains interpretable across conditions.

C.1 HYPOTHESES TESTING FRAMEWORK

We state explicit hypotheses for the three research questions in Sec. 3.

Research Q1: Do adversarial perturbations exploit superposition geometry?

- H_0 : Adversarial perturbations are random with respect to feature geometry.
- H_1 : Adversarial perturbations systematically exploit geometric relationships between superposed representations.

Research Q2: Do data correlations determine superposition geometry?

- H_0 : Input correlations have no systematic effect on learned geometries.
- H_1 : Input correlations determine geometric arrangements across model initializations.

Research Q3: Does shared geometry explain attack transferability?

- H_0 : Attack transferability is independent of geometric similarity.
- H_1 : Transferability increases with shared latent structure.

We test these hypotheses with controlled experiments:

- $H_1(1)$: We measure the input perturbation profile alignment with a class’s latent representation and the latent attack vector.
- $H_1(2)$: We systematically vary input correlations and measure resulting geometries.
- $H_1(3)$: We quantify transferability rates across models with varying geometric similarity.

C.2 ACCURACY OF TOY MODEL FOR A RANGE OF PARAMETERS

The main paper uses low-dimensional toy settings for conceptual clarity. Here we show that the same modelling behaviour extends across a broader parameter range by reporting classification accuracies for the CE toy model. We vary hidden layer size (h), number of classes (k), number of features, and sparsity (S) to identify when the models learn superposed feature representations. The sparsity level is the probability that any individual input feature $x_i^{(j)}$ is set to zero, so larger S means sparser inputs. We report results by feature density $1 - S$, the probability that a feature is non-zero. Tab. 2 and Tab. 3 establish the model’s performance limits and its ability to learn latent representations in superposition.

C.3 CORRELATIONS CONSTRAIN ARRANGEMENTS

Tab. 4 extends the *Do Input Correlations Determine Latent Feature Geometry?* experiments across a wider range of classes (k) and hidden dimensions (m), confirming that the correlation–geometry relationship is not confined to the smallest setting.

C.4 HOW THE TOY MODEL MAPS TO REAL-WORLD MODELS

Scaling changes adversarial vulnerability but does not eliminate the representational mechanism. Empirical evidence suggests that larger models tend to be somewhat more robust to adversarial attacks, although the effect is weak. With adversarial training, scaling trends become clearer, but the gains remain largely specific to the attack type used during training rather than producing general robustness (Howe et al., 2025).

Table 2: Classification accuracy of the CE toy model with a fixed bottleneck dimension ($m = 2$) across various numbers of classes (k), total input features (features = $k \times 3$), and input feature densities ($1 - S$). These results illustrate how input sparsity controls performance degradation as the number of classes to be superposed within a constrained latent space increases.

Classes (k)	Features	Hidden (m)	Accuracy at Input Feature Density ($1 - S$)							
			1.0	0.57	0.33	0.19	0.11	0.06	0.04	0.02
3	9	2	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
4	12	2	0.67	0.71	0.82	0.92	0.98	0.99	1.00	1.00
5	15	2	0.53	0.50	0.65	0.77	0.89	0.95	0.98	0.99
6	18	2	0.40	0.43	0.51	0.66	0.82	0.93	0.97	0.99
7	21	2	0.34	0.34	0.40	0.53	0.73	0.87	0.95	0.98
8	24	2	0.30	0.30	0.33	0.40	0.63	0.82	0.93	0.97
9	27	2	0.24	0.26	0.30	0.35	0.57	0.75	0.89	0.96
10	30	2	0.22	0.24	0.26	0.31	0.50	0.72	0.87	0.95
15	45	2	0.14	0.15	0.16	0.17	0.25	0.40	0.65	0.86
20	60	2	0.10	0.10	0.11	0.13	0.15	0.24	0.44	0.76
25	75	2	0.07	0.08	0.09	0.10	0.12	0.16	0.26	0.62
30	90	2	0.06	0.07	0.07	0.07	0.09	0.11	0.21	0.45

Table 3: Classification accuracy of the CE toy model for varying numbers of classes (k), total input features, bottleneck dimensions (m), and input feature sparsity levels (S). This table reports different numbers of features per class (p).

Classes (k)	Features	Hidden (m)	Accuracy at Input Feature Density ($1 - S$)							
			1.0	0.57	0.33	0.19	0.11	0.06	0.04	0.02
30	30	30	0.23	0.24	0.38	0.62	0.83	0.94	0.99	1.00
30	90	90	0.27	0.27	0.33	0.41	0.51	0.67	0.85	0.94
40	40	30	0.67	0.54	0.73	0.77	0.88	0.96	0.99	0.99
40	120	30	0.71	0.64	0.65	0.72	0.73	0.79	0.89	0.96
60	60	10	0.05	0.07	0.12	0.25	0.47	0.73	0.90	0.97
60	180	10	0.08	0.10	0.13	0.17	0.22	0.32	0.52	0.75
80	80	30	0.15	0.17	0.23	0.41	0.63	0.79	0.91	0.98
80	240	30	0.23	0.22	0.31	0.41	0.48	0.53	0.66	0.80
100	100	10	0.03	0.04	0.05	0.10	0.21	0.43	0.69	0.87
100	300	10	0.04	0.05	0.06	0.09	0.12	0.15	0.25	0.45

Table 4: Geometric similarity across correlation types and superposition configurations, showing that stronger correlations produce more consistent feature geometries.

Correlation Type	k	m	Geometric Similarity
Uncorrelated	6	2	0.18 ± 0.15
	30	10	0.17 ± 0.02
	90	30	0.17 ± 0.01
Paired	6	2	0.47 ± 0.07
	30	10	0.26 ± 0.07
	90	30	0.25 ± 0.01
Global	6	2	0.92 ± 0.04
	30	10	0.88 ± 0.01
	90	30	0.80 ± 0.01

Two forces act simultaneously as models scale. Larger models have more capacity to represent concepts, but general tasks such as next-token prediction over internet text also contain a long tail of useful concepts to encode. As a result, increased capacity does not remove superposition; superposition appears to remain prevalent even in frontier models (Lindsey et al., 2025). Dictionary-learning evidence supports this view: larger models require increasingly large SAE dictionaries (Gao et al., 2025), indicating that feature count scales with model size. The core tension driving superposition – compressing many features into limited dimensions – therefore persists under scale.

Because both superposition and adversarial vulnerability persist in large-scale models, the mechanism studied here remains relevant beyond toy settings. A key future direction is to determine how superposition geometry changes with scale and whether those geometric changes can be used to reduce vulnerability.

C.4.1 SEPARATING SUPERPOSITION EFFECTS FROM CAPACITY REDUCTION

Controlling for capacity by keeping the bottleneck dimension fixed to isolate superposition effects controls for the confounding effect of superposition pressure and capacity reduction. We fix bottleneck dimension $m = 2$, and vary number of classes k to isolate superposition pressure. We report the model accuracy, robust accuracy, and transferability in Tab. 5. These controls demonstrate that increased superposition pressure (k/m), independent of model capacity, drives the adversarial vulnerabilities we observe.

Table 5: Performance metrics across correlation types, perturbation budgets (ε), and number of classes (k)

Correlation Type	Parameters		Performance (%)		
	ε	k	Accuracy	Robust Acc.	Attack Trans.
Uncorrelated	0.05	2	100.0	100.0	0.0
		4	100.0	94.9	21.2
		6	99.9	87.0	9.8
		8	99.8	77.7	17.5
	0.1	2	100.0	100.0	0.0
		4	100.0	88.1	9.8
		6	99.7	75.4	9.9
		8	99.6	56.3	23.5
	0.5	2	100.0	100.0	0.0
		4	100.0	99.4	0.0
		6	99.9	62.9	30.5
		8	99.5	24.9	39.0
	0.05	2	100.0	100.0	0.0
		4	100.0	95.8	98.4
		6	97.3	70.6	98.4
		8	93.0	57.4	97.2
Fully Correlated	0.1	2	100.0	100.0	0.0
		4	100.0	88.7	98.5
		6	97.7	45.6	99.7
		8	94.2	34.8	98.6
	0.5	2	100.0	100.0	0.0
		4	100.0	90.0	100.0
		6	97.2	34.6	100.0
		8	93.0	32.0	100.0

C.4.2 ADDITIONAL EXAMPLES OF AEXS IN TOY MODEL

Fig. 1 demonstrates how adversarial attacks exploit the interference between latent features in superposition. Here we provide further visual examples (Fig. 5) to reinforce intuition. Specifically, we supplement the main text by showcasing:

- Fig. 5a shows an additional instance of the setup in Sec. 3.1 ($m = 2, k = 7$) with an ℓ_2 -norm PGD attack, demonstrating the IPP and latent space manipulations that lead to misclassification.
- Fig. 5b shows a similar setup ($m = 2, k = 7$) using an ℓ_∞ -norm PGD attack on a less sparse input.
- Fig. 5c shows an example with increased bottleneck dimensionality ($m = 3, k = 7$) and accompanying ℓ_2 -norm PGD attack. The feature vector similarity matrix used to calculate geometric similarity is also shown.

D CIFAR-10 EXPERIMENTS

To investigate whether the principles observed in the toy models extend to more complex settings, Sec. 4 introduces experiments trained on CIFAR-10 (Krizhevsky, 2009) with an engineered bottleneck. This appendix section provides further details on this setup and presents extended results.

D.1 ARCHITECTURE & TRAINING INFORMATION

The base ViT (Dosovitskiy et al., 2020) architecture comprised 6 transformer layers, an embedding dimension (d) of 512, and 8 attention heads in each transformer layer. Input images from the CIFAR-10 dataset, sized at 32×32 pixels, were processed into patches of 4×4 pixels. The Multilayer Perceptron (MLP) within each transformer block had a hidden dimension of 512. Learned positional embeddings were used.

The bottleneck architecture consisted of a linear encoder followed by a linear decoder. The linear encoder projected the pre-classification activations obtained from the ViT backbone into an m -dimensional latent space ($\{2, 3, 5, 10\}$). The subsequent linear decoder then mapped these m -dimensional representations back to the $k = 10$ dimensions corresponding to the CIFAR-10 classes.

The base ViT was trained on CIFAR-10 for 250 epochs using Adam with default PyTorch parameters, learning rate 0.001, batch size 512, cosine annealing, CE loss, and dropout. We trained across five random seeds to measure variability. After base training, we froze the ViT weights and trained only the bottleneck layer for 30 epochs with learning rate 0.001. Preprocessing used RandomCrop with 4-pixel padding, resizing to the target size (32x32 by default), and RandomHorizontalFlip for data augmentation.

D.1.1 NORMALISED ROBUST ACCURACY ACROSS PERTURBATION MAGNITUDES

Fig. 4 (right) reports normalised robust accuracy versus bottleneck dimension for one ϵ . This subsection broadens the result by reporting normalised robust accuracy under PGD attacks of multiple strengths. We include both ℓ_2 -norm (Tab. 7) and ℓ_∞ -norm (Tab. 6) PGD attacks, giving a fuller picture of how superposition degree and attack strength jointly determine robustness.

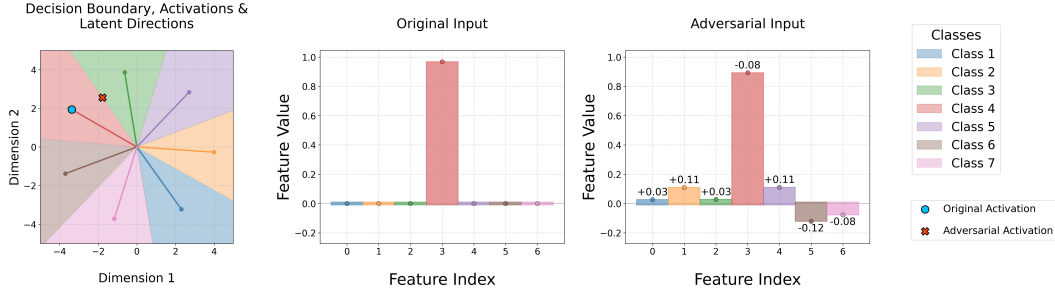
For these evaluations, we generated AExs using PGD with 100 iterations and step size (α) 0.01. Robust accuracy was evaluated on 500 samples for each configuration, and we report the mean normalised robust accuracy with standard deviation across five random seeds.

D.1.2 ATTACK TRANSFERABILITY ACROSS PERTURBATION MAGNITUDES

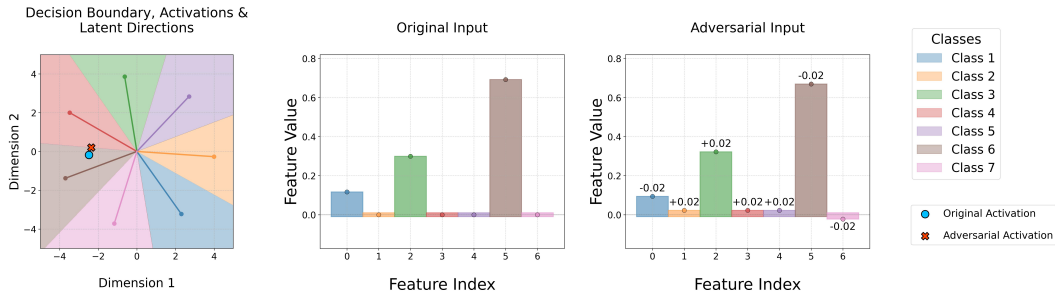
We report attack transferability across perturbation magnitudes (ϵ) and bottleneck dimensions (m). Tab. 9 gives ℓ_2 -norm attack transferability, and Tab. 8 gives ℓ_∞ -norm attack transferability.

D.2 CORRELATIONS IN CLASS FEATURES

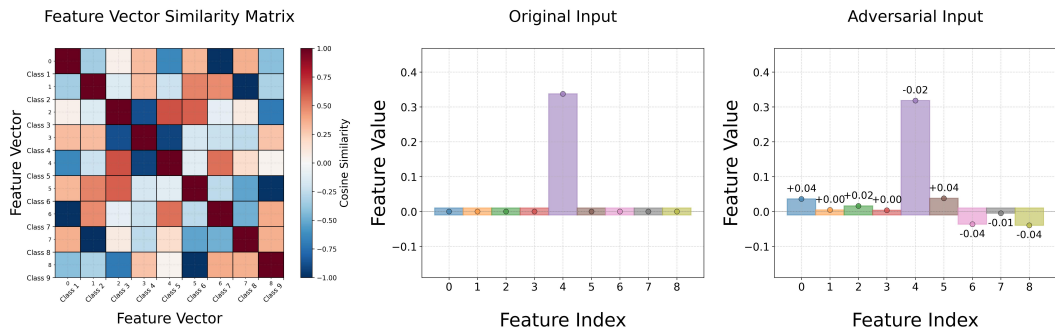
In Sec. 3, correlations between inputs drove superposition arrangements. In the ViT bottleneck, the relevant correlations are instead correlations in the representations arriving at the classification layer. These correlations are likely related to the confusion matrix: classes that cluster are those the network



(a) An ℓ_2 -norm attack changing the classification of an input of class 4 to class 3. The left plot shows original and adversarial activations in latent space, along with representation directions. The right plots show original and perturbed input feature values.



(b) An ℓ_∞ -norm attack changing the classification of an input of class 6 to class 4. The left plot shows original and adversarial activations in latent space relative to class latent directions. The right plots show original and perturbed input feature values.



(c) An ℓ_2 -norm adversarial attack in a 7-class setup with an increased bottleneck dimension $m = 3$. The leftmost plot now shows the cosine similarity matrix between pairs of latent representations for each of the classes.

Figure 5: Visualisations of AExs in the toy model, supplementing Figure 1 from the main paper by illustrating attack mechanisms in activation space and input space under varied conditions.

Table 6: Mean normalised robust accuracy ($\pm$ standard deviation across 5 seeds) for CIFAR-10 ViT models with different bottleneck dimensions (m), under ℓ_∞ -norm PGD attacks of varying perturbation magnitude (ϵ). Robust accuracy is normalised by each bottlenecked model’s clean accuracy, and the trend supports Sec. 4 across ϵ .

ϵ	Bottleneck Dimension (m)			
	2	3	5	10
0.001	96.0% $\pm$ 0.4%	97.4% $\pm$ 0.7%	97.9% $\pm$ 0.4%	98.1% $\pm$ 0.3%
0.01	61.7% $\pm$ 3.6%	69.6% $\pm$ 3.5%	77.0% $\pm$ 0.9%	81.8% $\pm$ 3.2%
0.05	4.9% $\pm$ 1.5%	6.7% $\pm$ 1.6%	9.3% $\pm$ 0.6%	10.5% $\pm$ 0.8%
0.1	0.1% $\pm$ 0.2%	0.2% $\pm$ 0.3%	0.2% $\pm$ 0.4%	0.2% $\pm$ 0.4%
0.5	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%	0.0% $\pm$ 0.0%

Table 7: Mean normalised robust accuracy ($\pm$ standard deviation across 5 seeds) for CIFAR-10 ViT models with different bottleneck dimensions (m), under ℓ_2 -norm PGD attacks of varying perturbation magnitude (ϵ). Robust accuracy is normalised by each bottlenecked model’s clean accuracy; robustness decreases with smaller m (increased superposition) and larger ϵ .

ϵ	Bottleneck Dimension (m)			
	2	3	5	10
0.1	90.4% $\pm$ 1.7%	91.8% $\pm$ 0.7%	94.6% $\pm$ 1.2%	95.0% $\pm$ 0.7%
0.5	58.5% $\pm$ 5.0%	60.8% $\pm$ 5.0%	69.2% $\pm$ 1.8%	72.6% $\pm$ 2.8%
1.0	41.7% $\pm$ 4.8%	44.0% $\pm$ 3.0%	50.4% $\pm$ 0.6%	54.9% $\pm$ 1.7%
2.0	34.0% $\pm$ 4.6%	36.2% $\pm$ 3.3%	41.5% $\pm$ 1.3%	47.4% $\pm$ 2.4%
5.0	30.2% $\pm$ 4.1%	32.9% $\pm$ 3.5%	37.3% $\pm$ 1.9%	42.7% $\pm$ 1.0%

treats as similar and tends to confuse. To test this interpretation, we repeat the experiment by fine-tuning a base ViT from timm/vit_base_patch16_384, pretrained on ImageNet-21k (14 million images, 21,843 classes) and ImageNet (1 million images, 1,000 classes). After fine-tuning on CIFAR-10 and applying the same bottleneck training procedure as Sec. 4, the geometry shows random ordering between initialisations with approximately equal feature spacing (*i.e.* neural collapse Kothapalli (2023)). Because performance is near 100%, the confusion matrix is close to the identity and the superposed geometry loses its structured class clusters. By contrast, models trained only on CIFAR-10 converge to similar geometries because they share the same learned difficulty structure: the same class pairs remain hard to distinguish, producing consistent superposition patterns.

D.2.1 RESNET-18

To address concerns on architectural generalisation, we conduct the same experiments using a ResNet-18 (He et al., 2016) architecture as the base model as opposed to a ViT. We achieve a slightly higher 92% clean accuracy (compared to 89% for ViT). We observe the robust accuracy falls slightly faster for ResNet-18 than the ViT at the same ϵ values. Nevertheless, we observe similar declining trends in normalised robust accuracy and increasing transferability as bottleneck pressure increases.

E MODULAR ADDITION

Sec. 5 introduces a modular addition task to investigate adversarial vulnerability in a scenario where features are orthogonal. This section provides a detailed experimental setup for this task, additional results and further discussion on how we view this sort of analysis to be useful.

Table 8: Attack transferability (%) for ℓ_∞ -norm PGD attacks on CIFAR-10 ViT models. Transferability is shown from a model trained with a specific ‘Source Seed’ (e.g., Seed 10) to three different target models, each trained with one of the seeds listed in the sub-header (e.g., ‘vs. Seeds 20/30/40’). The three slash-separated values in each cell correspond to the transferability to these three target seeds, respectively. All models within a row share the same bottleneck dimension, m . The ‘Mean $\pm$ Std’ column averages transferability across all 12 source-target seed pairings for each (ϵ, m) configuration. This supports the claim in Section 4 that higher superposition can lead to more consistent latent geometries and thus higher transferability.

ϵ	m	Seed 10 vs. Seeds 20/30/40	Seed 20 vs. Seeds 10/30/40	Seed 30 vs. Seeds 10/20/40	Seed 40 vs. Seeds 10/20/30	Mean $\pm$ Std
0.001	2	77.8/66.7/44.4	25.0/62.5/25.0	72.7/72.7/72.7	58.3/75.0/58.3	59.3 $\pm$ 17.7
	3	57.1/28.6/28.6	14.3/28.6/28.6	63.6/72.7/72.7	42.9/42.9/42.9	43.6 $\pm$ 18.4
	5	70.0/50.0/50.0	57.1/71.4/57.1	50.0/25.0/25.0	16.7/50.0/33.3	46.3 $\pm$ 16.9
	10	55.6/55.6/66.7	12.5/37.5/25.0	16.7/33.3/16.7	28.6/71.4/28.6	37.3 $\pm$ 19.3
0.01	2	60.3/52.6/48.7	63.8/42.0/60.9	53.8/40.9/52.7	51.2/55.8/46.5	52.4 $\pm$ 6.9
	3	42.9/46.4/50.0	46.4/42.9/49.1	44.2/38.9/44.2	42.9/45.1/41.8	44.6 $\pm$ 3.0
	5	43.3/41.1/43.3	39.2/30.4/41.8	46.0/40.2/47.1	41.2/35.3/32.9	40.2 $\pm$ 4.8
	10	42.5/46.0/44.8	38.0/39.4/46.5	43.5/47.8/47.8	32.6/40.0/35.8	42.1 $\pm$ 4.7
0.05	2	35.0/36.4/37.8	38.8/36.0/46.3	37.4/31.3/41.7	42.0/42.0/40.7	38.8 $\pm$ 3.8
	3	29.9/29.9/37.1	30.4/30.1/35.6	30.2/30.2/35.1	30.1/31.9/34.4	32.1 $\pm$ 2.6
	5	26.8/25.3/29.2	25.2/21.5/26.4	28.0/28.0/26.5	26.0/26.6/21.9	25.9 $\pm$ 2.2
	10	27.1/24.9/30.4	21.6/23.0/28.4	25.3/28.1/30.6	21.0/24.9/20.1	25.4 $\pm$ 3.4
0.1	2	34.2/35.6/38.2	36.7/34.9/44.5	37.9/30.0/40.8	41.2/42.0/40.3	38.0 $\pm$ 3.8
	3	29.6/28.9/35.2	28.6/29.2/34.8	29.1/28.2/34.0	28.8/31.1/32.8	30.9 $\pm$ 2.5
	5	25.4/23.3/27.0	23.8/19.6/24.6	25.9/26.7/25.4	24.6/24.3/20.7	24.3 $\pm$ 2.1
	10	24.8/22.6/29.7	20.1/21.6/26.1	22.5/25.5/28.0	19.5/23.3/18.7	23.5 $\pm$ 3.3

E.1 LEARNED ALGORITHMIC REPRESENTATION IN MODULAR ADDITION

As detailed in Nanda et al. (2023) and Prieto et al. (2025b), the network discovers and implements a trigonometric algorithm to solve this task: the encoder projects inputs onto non basis-aligned linear representations encoding $\sin(\omega_k a)$ and $\cos(\omega_k a)$ at key frequencies $\omega_k = 2\pi k/P$. These features are represented orthogonally, *i.e.* not in superposition, and the sinusoidal values can be read off via linear probes. The subsequent MLP then applies sum-angle identities to compute $\cos((a+b)\omega_k)$ for various k , which constructively interfere at the logits to select the correct class. AExs are generated by relaxing the one-hot constraint and perturbing input $\mathbf{x}_j \in \mathbb{R}^P$ (encoding integer j) to $\mathbf{x}' = \mathbf{x} + \delta$ via ℓ_2 and ℓ_∞ -norm PGD. For simplicity, only a is attacked while b is fixed.

As explained in Nanda et al. (2023), the network learns a structured trigonometric algorithm. W_e encodes integers’ sine and cosine values for key frequencies. These features are linearly decodable sine/cosine values. Scaling the corresponding feature vector alters the encoded sinusoidal value itself, not its activation intensity. Integer embeddings $\mathbf{W}_e^j$ project onto a Fourier-like basis, creating a circular/periodic structure in the embedding space. Unlike arrangements from data correlations, this circularity arises from the sine/cosine encoding of the (non-correlated) input integers.

Our model learns to utilise approximately seven key frequencies (ω_k). Each embedding W_e^j for an input integer x_j predominantly encodes its fourteen corresponding trigonometric features: $\cos(\omega_i x_j)$ and $\sin(\omega_i x_j)$ for $i \in \{1, 2, \dots, 7\}$. The concatenated 200-dim. vector thus provides these ≈ 28 key trigonometric features (fourteen for a and fourteen for b) to the MLP. The MLP leverages these, via sum-angle identities (e.g., $\cos(A+B) = \cos A \cos B - \sin A \sin B$), to calculate $(x_1 + x_2) \pmod{P}$. The algorithmic brittleness explored in Section 5 stems from the adversarial perturbation of these key learned trigonometric features.

Table 9: Attack transferability (%) for ℓ_2 -norm PGD attacks on CIFAR-10 ViT models. The table format, detailing source model to target model transferability, mirrors that of Tab. 8; please see its caption for a full explanation. These ℓ_2 results further support the claim in Sec. 4 that higher superposition leads to increased attack transferability.

ϵ	m	Seed 10 vs. Seeds 20/30/40	Seed 20 vs. Seeds 10/30/40	Seed 30 vs. Seeds 10/20/40	Seed 40 vs. Seeds 10/20/30	Mean $\pm$ Std
0.1	2	70.6/64.7/41.2	60.0/50.0/60.0	64.3/60.7/57.1	48.1/63.0/51.9	57.6 ± 8.0
	3	39.1/39.1/47.8	50.0/60.7/50.0	48.1/55.6/51.9	48.0/28.0/40.0	46.5 ± 8.4
	5	43.5/34.8/43.5	31.6/31.6/42.1	42.9/35.7/21.4	30.4/39.1/34.8	35.9 ± 6.4
	10	45.5/59.1/45.5	19.0/28.6/28.6	37.5/43.8/43.8	23.8/52.4/38.1	38.8 ± 11.4
0.5	2	59.6/52.1/48.9	60.8/35.4/60.8	54.1/39.4/51.4	43.8/49.5/46.7	50.2 ± 7.7
	3	38.7/40.6/48.1	36.1/36.8/41.7	37.4/36.5/43.5	34.4/34.4/36.7	38.7 ± 3.9
	5	38.4/32.0/37.6	29.4/25.7/32.1	39.1/33.9/38.3	31.8/29.9/32.7	33.4 ± 4.0
	10	38.3/37.4/37.4	26.5/28.4/31.4	32.4/35.3/35.3	23.4/28.2/25.0	31.6 ± 5.0
1.0	2	50.8/43.7/44.4	51.7/36.4/50.0	46.1/32.9/44.7	42.7/46.7/45.3	44.6 ± 5.3
	3	35.7/36.9/41.7	31.8/36.4/42.6	33.7/32.0/37.3	34.9/32.0/35.5	35.9 ± 3.3
	5	32.1/28.9/33.2	30.9/23.0/30.9	35.9/31.5/33.7	27.7/28.2/27.1	30.3 ± 3.3
	10	32.3/32.3/38.5	23.7/27.7/29.9	29.1/32.0/34.3	22.2/25.6/21.1	29.1 ± 5.0
2.0	2	47.0/41.6/44.3	48.2/36.2/48.2	43.9/33.5/42.1	43.4/43.4/42.8	42.9 ± 4.2
	3	31.9/37.7/40.3	31.1/35.6/40.6	32.6/31.1/36.8	35.4/33.3/34.3	35.1 ± 3.2
	5	29.9/25.4/32.1	26.3/22.1/26.7	32.3/31.8/29.5	27.9/27.9/23.6	28.0 ± 3.2
	10	30.8/30.4/35.7	22.5/24.9/28.6	24.8/29.7/32.7	20.4/26.1/19.0	27.1 ± 4.9
5.0	2	44.7/46.7/40.8	45.3/37.3/48.7	42.7/33.9/41.5	42.3/42.3/40.6	42.2 ± 3.8
	3	32.3/35.8/40.8	31.3/32.6/39.9	31.0/30.0/33.8	35.0/33.0/33.0	34.1 ± 3.2
	5	30.6/24.1/28.6	28.3/20.9/24.8	31.7/31.3/27.4	28.1/29.0/23.2	27.3 ± 3.3
	10	29.8/27.4/32.7	19.7/24.1/28.9	24.9/29.9/31.2	18.8/25.8/19.2	26.0 ± 4.6

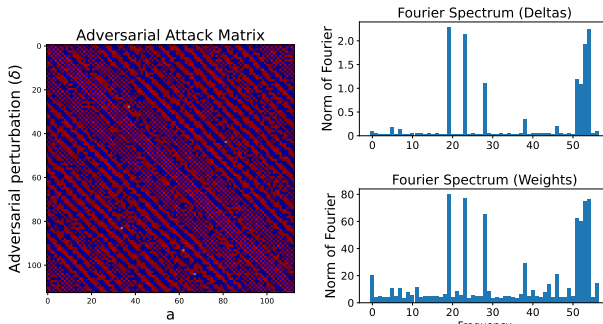


Figure 6: Left: Each column is the mean attack δ added to a to successfully attack $a + b \bmod p$. Right: The norms of the Fourier components in the δ and $W_{MLP_{out}}$.

Table 10: Comparison of minimum ℓ_2 -norm budget required to successfully attack $0 + 0 \bmod p$ between different attack types.

Attack Method	Success ϵ
PGD	0.17
Informed Attack	0.22
Single a	0.97
ℓ_∞ Noise	2.04
Uniform δ	6.20

E.2 EXPERIMENT DETAILS FOR MODULAR ADDITION TASK

The network is trained for addition modulo 113. Inputs are integer pairs (a, b) , $a, b \in \{0, \dots, 112\}$. Each integer is mapped to a 100-dimensional vector via a shared embedding $\mathbf{W}_E \in \mathbb{R}^{113 \times 100}$. Embeddings $\mathbf{W}_E^a$ and $\mathbf{W}_E^b$ are concatenated into a 200-dim. feature vector. This vector feeds a 3-hidden-layer MLP (200 ReLU neurons/layer). The output layer provides 113 logits for the

classification. The model is trained end-to-end using cross-entropy loss and AdamW (weight decay 4), achieving 100% test accuracy. AExs are generated using PGD under ℓ_2 and ℓ_∞ norms with varying ϵ . Attacks perturb the embedding $\mathbf{W}_E^a$ (with integer b fixed) by adding a perturbation δ to cause misclassification of $(a + b) \pmod{113}$.

E.3 ALGORITHMIC BRITTLINESS

The frequency-aligned perturbations cause misclassification by effectively altering the sinusoidal values read off from the encoder features. We use the hypothesised neuron-aligned case to give intuition: $\mathbf{W}_e$ would contain a row corresponding to the sine/cosine for each ω_k , whose element at column $\mathbf{w}_e^a$ contains the value $\sin(\omega_k a)$ or $\cos(\omega_k a)$. For the input a , the one-hot $\mathbf{x}_a$ acts to select $\mathbf{w}_e^a$, giving $\mathbf{h}_a = \mathbf{w}_e^a$. The perturbation δ transforms one-hot $\mathbf{x}_a$ into a dense vector $\mathbf{x}'_a$. Now $\mathbf{h}'_a = \mathbf{W}_e \mathbf{x}'_a = (1 + \epsilon_a) \mathbf{h}_a + \sum_{k \neq a} \epsilon_k \mathbf{w}_e^k$. The introduced contributions from the $k \neq a$ sinusoidal terms corrupts the sinusoidal values, thereby disrupting the MLP’s subsequent computations and leading to misclassification. The model’s algorithm uses the constructive interference of each ω_k at the logits, and as such disrupting a single ω_k is insufficient as the other components overwhelmingly support the correct answer. Thus, effective attacks must find δ that maximally alters multiple frequency components.

E.4 ANALYTIC CONSTRUCTION OF ATTACKS AND BASELINE COMPARISONS

The main paper describes how understanding the learned trigonometric algorithm in the modular addition task allows for the analytic construction of adversarial attacks. Sec. 5 of the main paper benchmarks this ‘informed attack’ against PGD and naive baselines. This section provides further details on these baseline attack methods. The baselines are:

- **PGD Attack:** Standard PGD used to find the adversary.
- **Informed Attack:** The analytically constructed perturbation, from combinations of cosines.
- **Single a :** This involved finding the minimal attack to a single input to cause misclassification.
- **ℓ_∞ Noise:** This baseline involved adding random noise δ_{noise} to the input a . Each component of δ_{noise} was sampled independently and uniformly from $[-\epsilon, \epsilon]$. The ϵ that successfully induced a misclassification is reported.
- **Uniform δ :** The perturbation vector δ is constrained to have all components equal, i.e., $\delta_j = c \cdot 1$, where c is a scalar. The magnitude of c is reported such that adding δ to the input causes a misclassification.

E.5 DETAILS ON CERTIFIED ROBUST TRAINING

To investigate the impact of defences on the identified vulnerability, we performed certified robust training. This aimed to make the network provably robust against ℓ_∞ -norm perturbations. The certified training method employed was an abstraction-based technique (Wong & Kolter, 2018; Raghunathan et al., 2018; Palma et al., 2022), namely RSIP-IBP (Zhang et al., 2020; Henriksen & Lomuscio, 2023). RSIP-IBP utilises Interval Bound Propagation (IBP) (Katz et al., 2017; Ehlers, 2017) and Reversed Symbolic Interval Propagation (RSIP) to compute reachable output sets for network layers, thereby enabling formal certification of robustness within a given perturbation budget. The training loss function incorporated both a standard loss component and an RSIP loss component. Certified training was conducted for 1000 epochs, with the initial 200 epochs serving as a warm-up phase. The learning rate was 1×10^{-3} and weight decay 1×10^{-4} . The same train/test data splits were used as for the standard model training. The results of certified training across different training perturbation budgets (ϵ_{train}) and evaluated against various attack perturbation budgets (ϵ_{attack}) are summarised in Tab. 11.

As discussed in Sec. 5, subsequent PGD attacks on these robustly trained models retained the same mechanistic character: successful perturbations continued to exploit the key frequency components of the learned algorithm. Certified training primarily increased the perturbation magnitude required for misclassification at certified epsilons; it did not fundamentally change the attack vector’s alignment with the model’s algorithmic sensitivities.

Table 11: Verified robust accuracy (%) for modular addition models under ℓ_∞ -norm perturbations. The table compares a standard (pre-robustification) model with models certifiably trained using RSIP-IBP with different training perturbation budgets (ϵ_{train}). Accuracies are evaluated against various attack budgets (ϵ_{attack}). These results support the discussion in Section 5.1 of the main paper, indicating that certified training improved robustness to specified ϵ_{train} but attacks still succeeded by exploiting the same algorithmic sensitivities at higher ϵ_{attack} .

Attack Epsilon (ϵ_{attack})	Train Epsilon (ϵ_{train})	Standard Acc. (Pre-Rob., %)	Robust Acc. (Pre-Rob., %)	Standard Acc. (Robust, %)	Robust Acc. (Robust, %)
5.0×10^{-4}	10^{-4}	100.0	100.0	100.0	100.0
	10^{-3}				100.0
1.0×10^{-3}	10^{-4}	100.0	76.0	100.0	93.8
	10^{-3}				100.0
2.0×10^{-3}	10^{-4}	100.0	0.0	100.0	0.0
	10^{-3}				95.7
5.0×10^{-3}	10^{-4}	100.0	0.0	100.0	0.0
	10^{-3}				85.3

E.6 ON A POTENTIAL RECONCILIATION WITH SUPERPOSITION VULNERABILITY

Although the vulnerability studied here arises without feature superposition, it shares an important structural feature with superposition-driven vulnerability: dependence on sparsity in the input representation. This motivates our *reconciliation conjecture*: algorithmic vulnerabilities can persist in networks with orthogonal representations when upstream sparsity assumptions are violated. If earlier layers produce noisy or non-sparse features, potentially through adversarial interference, those deviations can propagate into otherwise isolated downstream circuits. Circuits that compute correctly on clean inputs may then fail because their inputs are effectively pre-perturbed. This gives a plausible route by which algorithm-disruption mechanisms, including the frequency attacks studied here, can be activated even without local superposition. The interaction between upstream superposition and downstream algorithmic brittleness, and more generally the layer-by-layer mechanisms triggered by AExs, remains underexplored and warrants direct empirical and theoretical study.

F INSPECTING ADVERSARIAL PERTURBATIONS VIA SAE FEATURES

The main paper analyses settings with explicitly defined class features (toy model) or class-level bottleneck representations (ViT experiments). In large practical networks, however, interference occurs among unlabelled latent features across many layers. As discussed in the main paper’s future work section, SAEs provide a promising unsupervised route for extracting more linear features from NNs (Bricken et al., 2023). This appendix presents a preliminary but concrete investigation of whether SAE-extracted features can characterise how a large-scale ViT responds internally to AExs compared with clean inputs. The goal is feasibility and direction-setting rather than a definitive new claim. We first describe the setup, then validate SAE behaviour on adversarial inputs, and finally present exploratory layer-wise feature-difference analyses.

F.1 EXPERIMENT SETUP

We analyse SAE-extracted features from the vision encoder of LAION/CLIP-ViT-B-32-DataComp.XL-s13B-b90K, a 12-layer ViT with $\approx 70\%$ zero-shot accuracy on ImageNet-1k Deng et al. (2009). We use open-source Prisma SAEs that map the 768-dimensional activation space to a dictionary of 49,152 features ($64\times$ expansion) (Joseph et al., 2025). We examine SAEs trained on either mean patch-token representations or CLS-token representations across all layers. We analyse features activated above a high threshold (0.1), capturing the strongest activations, and a low threshold (0.001), capturing broader feature changes. We generate 10,000 original/successfully attacked image

pairs (ℓ_∞ -norm, $\epsilon = 0.02$) for the SAE feature comparisons. This setup gives a first controlled view of how adversarial perturbations appear at the level of SAE features.

F.2 SAE VALIDITY FOR ADVERSARIAL INPUTS

Because AExs are out-of-distribution inputs and the SAEs are standardly trained, we first validate that the SAEs remain useful in this setting. We measure model accuracy and SAE reconstruction effects when SAEs are inserted at inference time for both clean and adversarial inputs. This checks whether the SAEs still process classification-relevant features under adversarial conditions before we interpret feature-activation differences.

Table 12: Impact on ImageNet-1k classification accuracy when inserting a pre-trained SAE (one layer at a time) during inference for original, ℓ_∞ adversarial ($\epsilon = 0.02$), and random noisy inputs. ‘Accuracy’ is ViT top-1 accuracy. ‘Original input + SAE’ means clean ViT activations at a layer are passed through the SAE, after which the reconstruction is passed to the next layer. The slight ‘de-attacking’ effect (‘Adv. input + SAE’ vs. ‘Adv. input’) suggests that SAEs can filter some adversarial noise, while the remaining misclassification indicates that salient adversarial changes are still processed.

Metric	Layer 2	Layer 5	Layer 8	Layer 11
Accuracy				
Original input	1.00	1.00	1.00	1.00
Original input + SAE	0.99	0.99	0.99	0.86
Adv. input	0.00	0.00	0.00	0.00
Adv. input + SAE	0.02	0.01	0.01	0.11
Noisy input	0.98	0.98	0.98	0.98
Noisy input + SAE	0.98	0.98	0.98	0.85
SAE Improvement				
Original input	-0.01	-0.01	-0.01	-0.14
Adv. input	0.02	0.01	0.01	0.11
Noisy input	-0.00	-0.00	-0.00	-0.13

F.3 LAYER-WISE SAE FEATURE DIFFERENCES

This subsection presents exploratory visualisations comparing SAE feature activations across ViT layers for original, ℓ_∞ adversarial, and random noisy inputs. These visualisations identify concrete feature-level signatures that future work can test more directly. We examine:

- **Activation count:** Histograms comparing the number of SAE features activated above a threshold in each layer for original, adversarial, and noisy images. This tests whether attacks characteristically alter feature sparsity.
- **Feature overlap:** The number of common activated features between (original vs. adversarial) and (original vs. noisy) conditions. This tests whether attacks activate a distinct feature set.
- **Jaccard Similarity:** The Jaccard similarity between sets of activated features, offering a normalised measure of overlap.
- **Mean activation:** Histograms of mean activation values for features active above a threshold. This tests whether attacks change the intensity of key features.
- **Distinct Features:** The count of unique features activated across an image set for each condition.

F.4 CLS TOKEN SAE FEATURES WITH HIGH ACTIVATION THRESHOLD (0.1)

We first analyse CLS-token SAE features activated above the high threshold 0.1. Figures 7 through 10 compare activation-count histograms, feature overlap, Jaccard similarity, and mean-activation histograms for original, adversarial, and random noisy inputs across ViT layers.

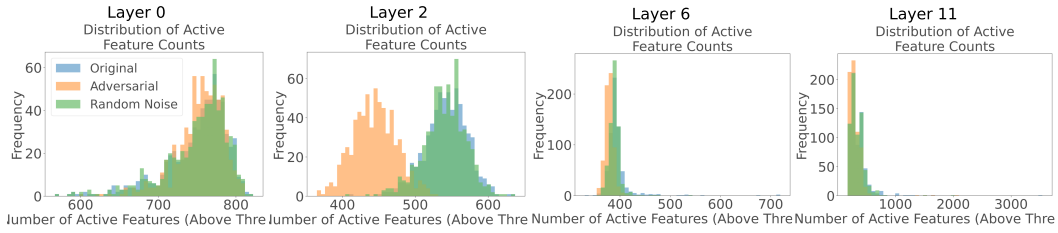


Figure 7: Activation-count histogram for high-threshold CLS-token SAE features (threshold 0.1).

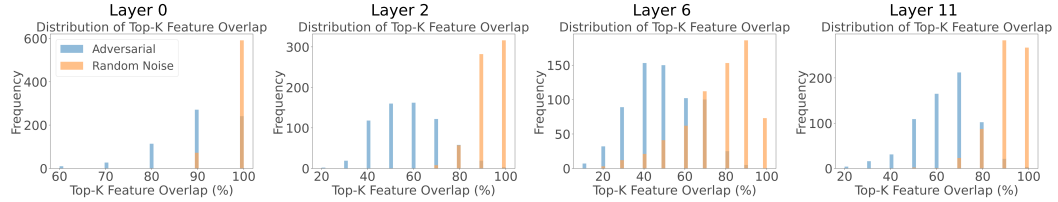


Figure 8: Feature overlap for high-threshold CLS-token SAE features, comparing original-attacked and original-noisy image pairs (threshold 0.1).

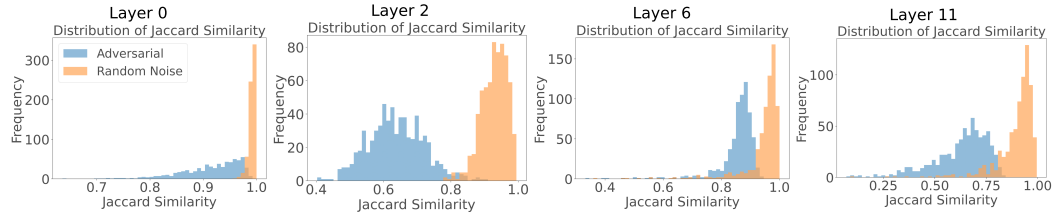


Figure 9: Jaccard similarity comparison for high-threshold CLS-token SAE features (threshold 0.1).

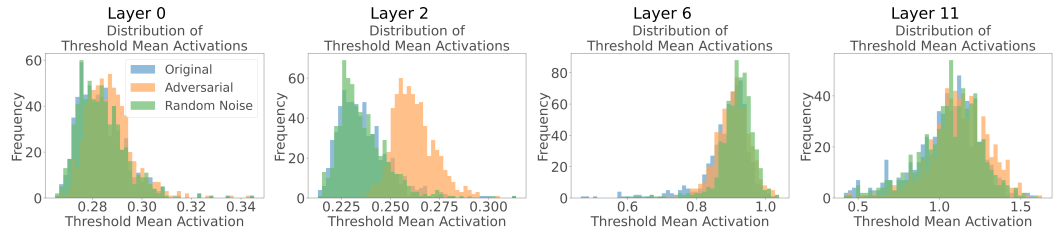


Figure 10: Mean-activation histogram for high-threshold CLS-token SAE features (threshold 0.1).

F.5 CLS TOKEN SAE FEATURES WITH LOW ACTIVATION THRESHOLD (0.001)

We then lower the CLS-token activation threshold to 0.001 to capture broader feature changes beyond the strongest activations. The figures report activation-count histograms (Fig. 11), feature overlap (Fig. 12), Jaccard similarity (Fig. 13), and mean-activation histograms (Fig. 14) across layers for original, adversarial, and noisy inputs.

F.6 PATCH TOKEN SAE FEATURES WITH HIGH ACTIVATION THRESHOLD (0.1)

We next analyse SAE features derived from mean patch-token representations using the high threshold 0.1. We compare activation-count histograms (Fig. 15), distinct-feature count histograms (Fig. 16), feature overlap (Fig. 17), Jaccard similarity (Fig. 18), and mean-activation histograms (Fig. 19) for original, adversarial, and noisy inputs.

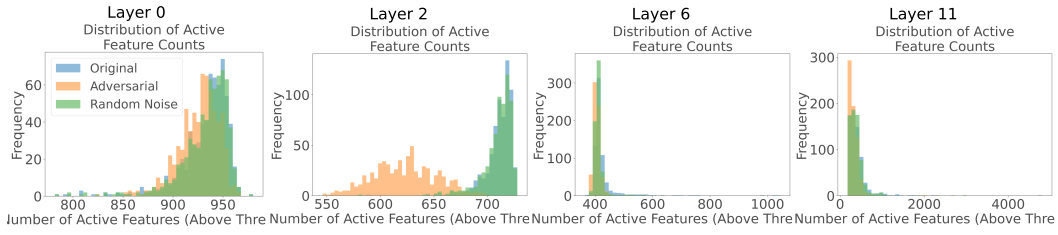


Figure 11: Activation-count histogram for low-threshold CLS-token SAE features (threshold 0.001), including noise.

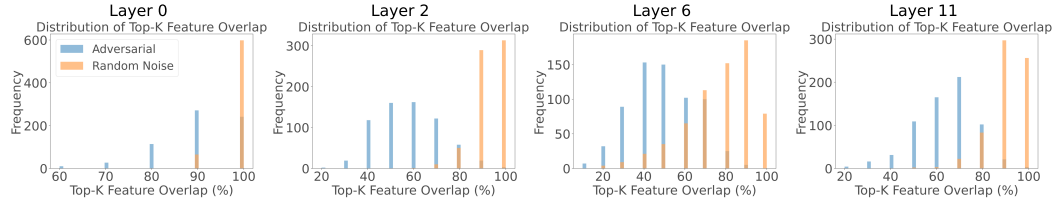


Figure 12: Feature overlap for low-threshold CLS-token SAE features, comparing original-attacked and original-noisy image pairs (threshold 0.001).

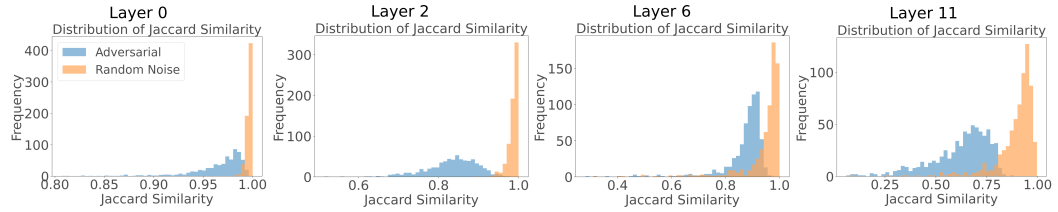


Figure 13: Jaccard similarity comparison for low-threshold CLS-token SAE features (threshold 0.001).

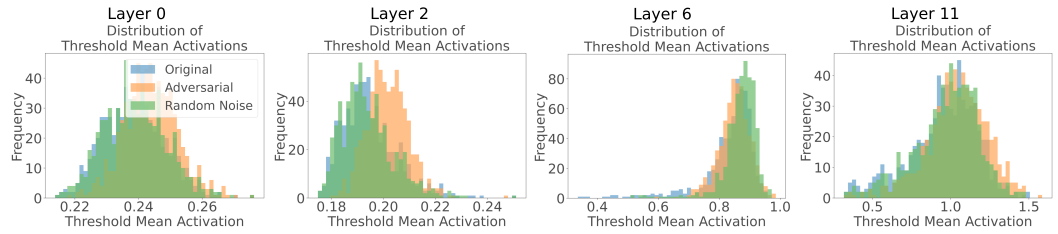


Figure 14: Mean-activation histogram for low-threshold CLS-token SAE features (threshold 0.001), including noise.

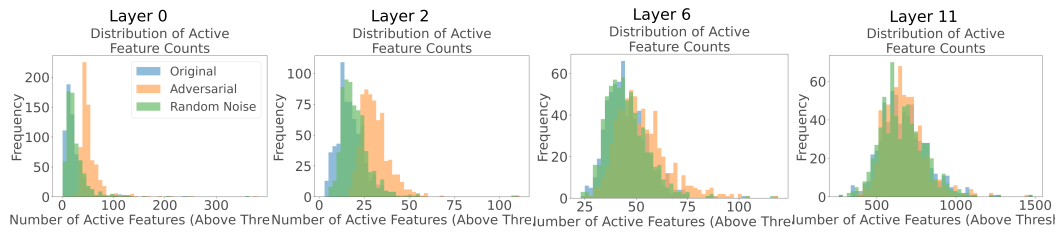


Figure 15: Activation-count histogram for high-threshold patch-token SAE features (threshold 0.1), including noise.

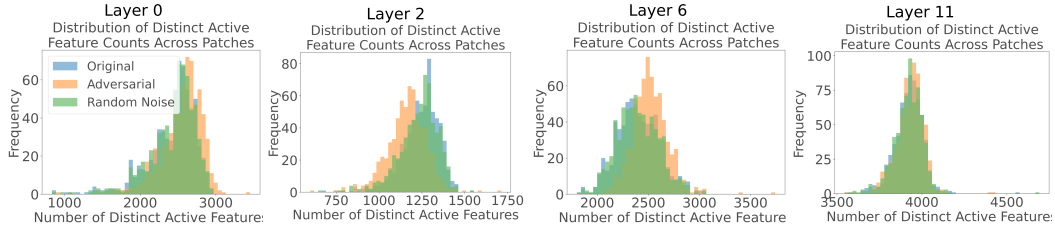


Figure 16: Distinct-feature histogram for high-threshold patch-token SAE features (threshold 0.1), including noise.

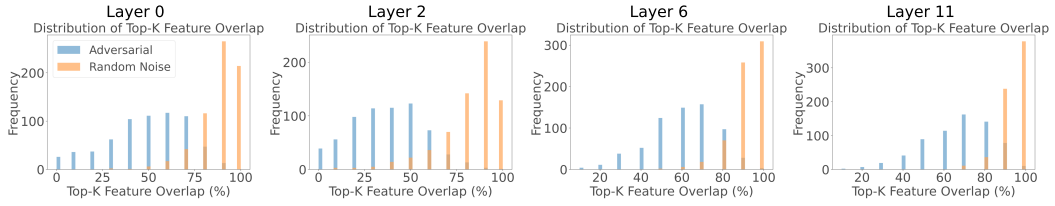


Figure 17: Feature overlap for high-threshold patch-token SAE features, comparing original-attacked and original-noisy image pairs (threshold 0.1).

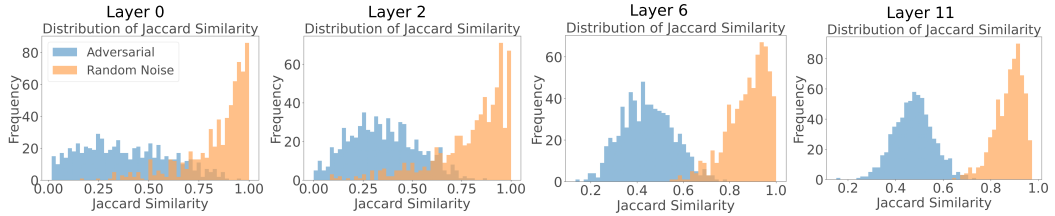


Figure 18: Jaccard similarity comparison for high-threshold patch-token SAE features (threshold 0.1).

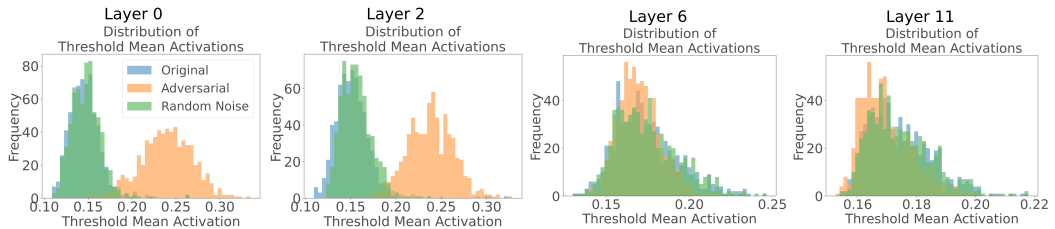


Figure 19: Mean-activation histogram for high-threshold patch-token SAE features (threshold 0.1), including noise.

F.7 PATCH TOKEN SAE FEATURES WITH LOW ACTIVATION THRESHOLD (0.001)

Finally, we analyse mean patch-token SAE features using the low threshold 0.001, capturing broader activation changes. The figures show activation-count histograms (Fig. 20), distinct-feature counts (Fig. 21), feature overlap (Fig. 22), Jaccard similarity (Fig. 23), and mean-activation histograms (Fig. 24) for original, adversarial, and noisy inputs.

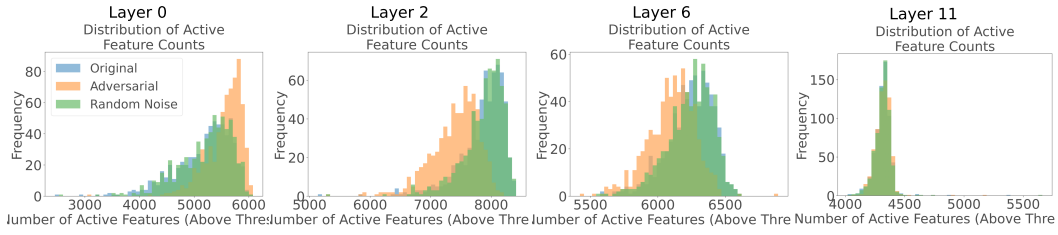


Figure 20: Activation-count histogram for low-threshold patch-token SAE features (threshold 0.001), including noise.

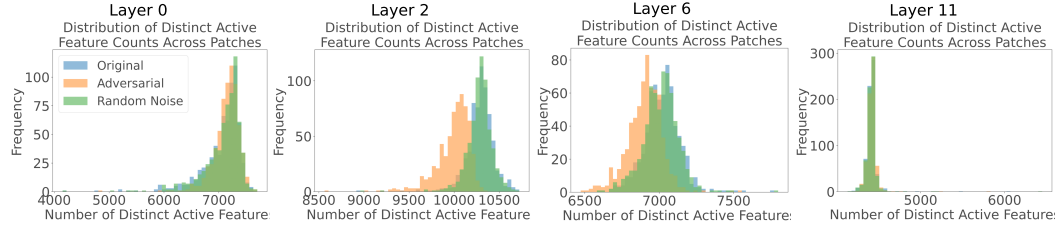


Figure 21: Distinct-feature histogram for low-threshold patch-token SAE features (threshold 0.001), including noise.

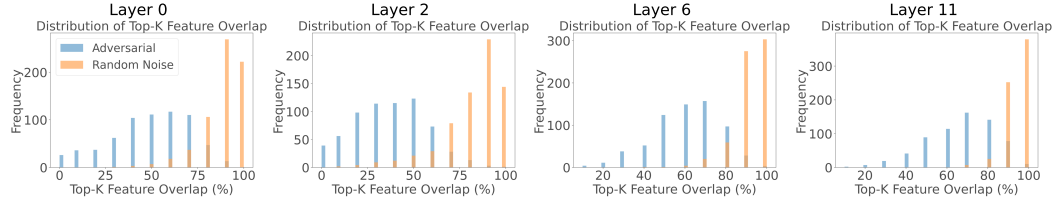


Figure 22: Feature overlap for low-threshold patch-token SAE features, comparing original-attacked and original-noisy image pairs (threshold 0.001).

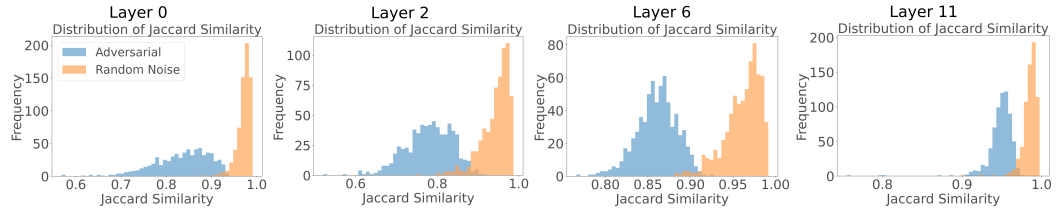


Figure 23: Jaccard similarity comparison for low-threshold patch-token SAE features (threshold 0.001).

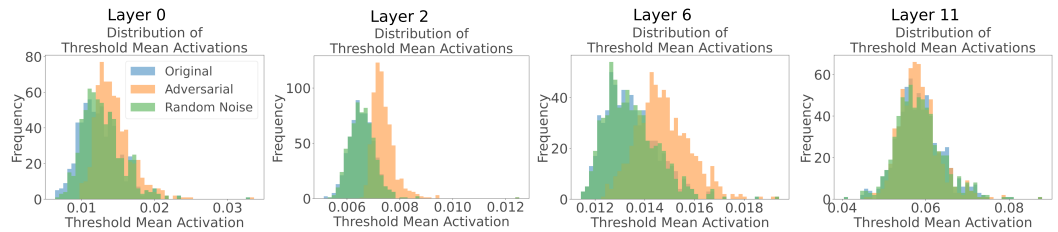


Figure 24: Mean-activation histogram for low-threshold patch-token SAE features (threshold 0.001), including noise.

1674 G ACRONYMS
1675
1676 **AEx** adversarial example
1677 **CE** cross entropy
1678 **LRH** linear representation hypothesis
1679 **MLP** Multilayer Perceptron
1680 **MSE** mean squared error
1681 **NN** neural network
1682 **PGD** projected gradient descent
1683 **SAE** sparse autoencoder
1684 **ViT** vision transformer
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727